

POLITECNICO
MILANO 1863

Best Bike Paths, Luca Suzani, Alberto
Spezzapria, Alessio Pesavento

Requirement Analysis and Specification Document

Deliverable:	RASD
Title:	Requirement Analysis and Verification Document
Authors:	Luca Suzani, Alberto Spezzapria, Alessio Pesavento
Version:	1.0
Date:	22-December-2025
Download page:	https://github.com/Alessio01103/PesaventoSpezzapriaSuzani.git
Copyright:	Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Contents

Table of Contents . . . 3
List of Figures . . . 5
List of Tables . . . 5

1 Introduction **7**
1.1 Purpose . . . 7
1.1.1 Goals . . . 7
1.2 Scope . . . 8
1.2.1 World Phenomena . . . 8
1.2.2 Shared Phenomena . . . 8
1.3 Definitions, Acronyms, and Abbreviations . . . 9
1.3.1 Definitions . . . 9
1.3.2 Acronyms . . . 9
1.3.3 Abbreviations . . . 10
1.4 Revision History . . . 10
1.5 Reference Documents . . . 10
1.6 Document Structure . . . 10
1.7 Use of Generative AI Tools . . . 10

2 Overall Description **12**
2.1 Product Overview . . . 12
2.1.1 Scenarios . . . 12
2.1.2 Domain Class Diagram . . . 15
2.1.3 State Diagrams . . . 16
2.2 Product Functions . . . 22
2.3 User Characteristics . . . 25
2.3.1 Registered User . . . 25
2.3.2 Unregistered User . . . 25
2.4 Assumptions, dependencies and constraints . . . 25
2.4.1 Assumptions . . . 25
2.4.2 Dependencies . . . 26
2.4.3 Constraints . . . 26

3 Specific Requirements **27**
3.1 External Interface Requirements . . . 27
3.1.1 User Interfaces . . . 27
3.1.2 Hardware Interfaces . . . 30
3.1.3 Software Interfaces . . . 30
3.1.4 Communication Interfaces . . . 30
3.2 Functional Requirements . . . 30
3.2.1 Use case diagrams . . . 33

3.2.2 Use cases . . . 34
3.2.3 Sequence diagrams . . . 40
3.2.4 Requirement Mapping . . . 46
3.3 Performance Requirements . . . 51
3.4 Design Constraints . . . 52
3.4.1 Standards Compliance . . . 52
3.4.2 Hardware Limitations . . . 52
3.4.3 Any other Constraint . . . 52
3.5 Software System Attributes . . . 52
3.5.1 Reliability . . . 52
3.5.2 Availability . . . 53
3.5.3 Security . . . 53
3.5.4 Maintainability . . . 53
3.5.5 Portability . . . 54

4 Formal Analysis Using Alloy **55**
4.1 Full Lifecycle Scenario . . . 57
4.1.1 Visual Analysis . . . 58
4.2 Conflicting Feedback Merge Scenario . . . 60
4.3 Rejected Activity Filter Scenario . . . 61

5 Effort Spent **62**

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

List of Figures

2.1 Domain Class Diagram 15
2.2 Activity Recording State Diagram 16
2.3 Path Creation State Diagram 17
2.4 Meteorological Data Retrieval State Diagram 19
2.5 Path Research State Diagram 20
2.6 User Lifecycle State Diagram 21

3.1 Public path search 27
3.2 Dashboard 28
3.3 Profile page 28
3.4 Activity recording interface 29
3.5 User's personal paths 29
3.6 Utilities 33
3.7 [UC1] User Registration Sequence Diagram 40
3.8 [UC2] System Access Sequence Diagram 41
3.9 [UC3] Activity Recording Sequence Diagram 42
3.10 [UC4] Creation of a manual path Sequence Diagram 43
3.11 [UC5] Publication of a path Sequence Diagram 44
3.12 [UC6] Path Feedback reporting Sequence Diagram 45
3.13 [UC7] Search for a path Sequence Diagram 46

4.1 Full Lifecycle Scenario Graph 59
4.2 Full Lifecycle Scenario Graph, Projection User0. 59
4.3 Alloy instance: Full Lifecycle Scenario Projection User1. 60
4.4 Conflicting Feedback Merge Scenario Graph 61
4.5 Rejected Activity Scenario Graph 61

List of Tables

3.2 Use Case: [User Registration] 34
3.4 Use Case: [System Access] 35
3.6 Use Case: [Activity Recording] 36
3.8 Use Case: [Creation of a Manual Path] 37
3.10 Use Case: [Publication of a Path] 38
3.12 Use Case: [Path Feedback reporting] 38
3.14 Use Case: [Search for a Path] 39
3.16 Requirement Mapping for G1 47
3.18 Requirement Mapping for G2 48
3.20 Requirement Mapping for G3 49
3.22 Requirement Mapping for G4 50
3.24 Requirement Mapping for G5 51

5.1 Hours of work spent by each author redacting the RASD document 63

Chapter 1

Introduction

1.1 Purpose

The purpose of the *Best Bike Paths (BBP)* system is to promote cycling as a sustainable, safe, and conscious means of transportation by providing cyclists with a digital platform for collecting, accessing, and sharing information about cycling routes and activities.
The system addresses the need for reliable and up-to-date data on bike paths, allowing users to track their performance while contributing to the improvement of shared information.
Through active community participation, BBP aims to identify and maintain the safest and most efficient routes, fostering a more informed and widespread cycling mobility.
Ultimately, BBP seeks to enhance the overall cycling experience by integrating multiple functionalities and creating a collaborative, continuously updated information network driven by user contributions.

1.1.1 Goals

G1. **Support cyclists in tracking their activities:** Enable users to record their cycling routes and access detailed statistics such as total distance, average speed, and other performance indicators, enriched, when available, by meteorological data from external services (weather conditions, temperature and wind speed).

G2. **Allow cyclists to contribute information about bike paths:** Provide two modes for entering information related to bike paths:

- **Manual mode:** the user manually inputs the path data and its condition.
- **Automatic mode:** the system automatically collects data via GPS, accelerometer, and gyroscope during the cycling activity, requiring user confirmation before publication.

G3. **Enable the publication of recorded path information:** Allow users to make their recorded path information public, contributing to a shared and valuable database for the entire cycling community.

G4. **Enable route search and visualization:** Allow users to search for and display on a map one or more cycling routes connecting a chosen origin and destination, ranking them based on overall condition and effectiveness in reaching the goal.

G5. **Maintain up-to-date and reliable path information:** Ensure continuous updating of path data through the aggregation and analysis of user feedback, taking into account data freshness, consistency among reports, and user confirmations.

1.2 Scope

The main goal of the *Best Bike Paths (BBP)* system is to provide cyclists with a mobile application that allows them to record their activities and share information about bike routes, while also fostering the creation of an active and collaborative community. The application enables registered users to save their rides, view detailed statistics such as total distance covered or average speed, and contribute to updating the path inventory by reporting the condition of routes or the presence of potential obstacles. The information entered by users can be made public and is then aggregated by the system, which combines data from multiple sources to offer a comprehensive and up-to-date representation of the paths' conditions. Unregistered users can still take advantage of the consultation features by searching for routes between two points and viewing the available options on a map, ranked according to a score that considers both the quality and effectiveness of the routes. The system also interacts with an external meteorological service, which provides weather data to enrich the information related to recorded activities.

1.2.1 World Phenomena

WP1. Cyclists travel through real-world routes by following predefined paths.

WP2. Cyclists perceive the quality of the path.

WP3. Cyclists search for paths between two real-world points.

WP4. The physical condition of paths can vary over time.

WP5. Mobile devices can detect data about the cyclist in the real world.

WP6. Real-world weather conditions are detected by meteorological services.

WP7. There are bike lanes and roads where speed limits or the absence of cars make them suitable for bicycles.

1.2.2 Shared Phenomena

User-System

SP1. Users can register within the application.

SP2. The user can record and store their cycling activities.

SP3. The user can choose whether to publish route-related data.

SP4. The system can provide the user with statistics about paths.

SP5. The user can manually enter information about paths.

SP6. The system can automatically record information, and the user must confirm it.

SP7. The user searches for paths based on a starting point and an ending point.

System-Device

SP8. The system receives data from mobile devices to track routes and detect obstacles.

SP9. The system sends requests to an external service and receives weather data.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

1.3 Definitions, Acronyms, and Abbreviations

1.3.1 Definitions

- **Activity**: A recorded session of a cycling trip that includes temporal data, performance metrics, and environmental conditions.
- **Automatic Mode**: A feature that enables the system to autonomously record the route and detect obstacles using the device's sensors (GPS, accelerometer, gyroscope).
- **Guest User**: An unauthenticated user who can access the system with limited privileges, specifically restricted to searching and viewing public paths.
- **Manual Mode**: A mode that allows users to create a path by manually selecting streets and defining conditions on the map, without physical recording.
- **Merging**: The system process that aggregates feedback and data from multiple users to update the status and reliability of a public path.
- **Path**: The core entity representing a bike route, defined by a sequence of geographic coordinates and metadata. It can be private or public.
- **Registered User**: An authenticated user with a personal profile who can record activities, create paths, and contribute feedback to the community.

1.3.2 Acronyms

- **API**: Application Programming Interface.
- **BBP**: Best Bike Paths.
- **FaaS**: Function as a Service.
- **GDPR**: General Data Protection Regulation.
- **GNSS**: Global Navigation Satellite System.
- **GPS**: Global Positioning System.
- **GPX**: GPS Exchange Format.
- **HTTPS**: HyperText Transfer Protocol Secure.
- **ISO/IEC**: International Organization for Standardization / International Electrotechnical Commission.
- **OAuth**: Open Authorization.
- **RASD**: Requirement Analysis and Specification Document.
- **RDO**: Requirement Document (Reference to the Project Assignment).
- **SSL**: Secure Sockets Layer.
- **TLS**: Transport Layer Security.
- **UML**: Unified Modeling Language.
- **WGS84**: World Geodetic System 1984.
- **WSS**: WebSocket Secure.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

1.3.3 Abbreviations

- **DB**: DataBase.
- **UI**: User Interface.
- **UX**: User Experience.

1.4 Revision History

Version 1.0: 22/12/2025

1.5 Reference Documents

- **RDO:** Assignment RDD AY 2025-2026 - Software Engineering 2 Project Assignment, Politecnico di Milano.
- **UML:** OMG Unified Modeling Language (UML), Version 2.5.1.
- **Alloy:** Jackson, Daniel. "Software Abstractions: Logic, Language, and Analysis." MIT Press.
- **GDPR:** Regulation (EU) 2016/679 (General Data Protection Regulation).

1.6 Document Structure

The following list provides a brief overview of the contents of each chapter in this document:

- **Chapter 1 Introduction:** A brief description of the project that follows from the analysis of the specification provided; it highlights the goals of the product and its scope.
- **Chapter 2 Overall Description:** A more detailed description of the product that comprehends also real world scenarios and analyses the system from a technical point of view.
- **Chapter 3 Specific Requirements:** A detailed list of all the requirements needed in order to satisfy the goals, and their relationship with domain assumptions.
- **Chapter 4 Formal analysis using Alloy:** A formal description of a portion of real world phenomena analyzed through Alloy in order to highlight inconsistencies between assumptions and phenomena previously modelized.
- **Chapter 5 Effort spent:** The time spent by each member of the group to contribute to the realization of this document.

1.7 Use of Generative AI Tools

In compliance with the course guidelines regarding academic integrity and content responsibility, we declare that we have used Generative Artificial Intelligence (GenAI) tools to support the drafting of this document (RASD). We emphasize that the entire content of this document is the result of our original reasoning; AI was not used to generate domain logic or requirements from scratch, but exclusively as a tool for linguistic support and formal verification. The details of its usage are reported below:

1. **Tools Used**
    - **Tool Name:** ChatGPT (model GPT-3.5/4o mini), Gemini Advanced (Gemini 3).

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

2. **Provided Inputs (Prompts and Context)** The inputs provided to the system were exclusively textual and based on preliminary work carried out by the group. Specifically:

    - **For linguistic revision:** We input paragraphs or entire sections written by us in Italian/English, with the request to improve grammatical form, academic vocabulary, and text fluidity, without altering the technical meaning.

        Example A - For stylistic improvement: *"Act as a professional Technical Writer reviewing a Requirements Analysis and Specification Document (RASD). I will provide you with a raw text describing the 'User Registration' process. Please rewrite it to improve grammatical correctness and use formal academic English suitable for a software engineering document. Do NOT add new information or change the logical flow of the requirements. Just polish the language."*

    - **For completeness verification:** We provided lists of functional requirements or scenario descriptions developed by the group, asking the model to flag potential logical ambiguities, missing use cases, or inconsistencies regarding the standard RASD structure.

        Example B - For completeness verification: *"Act as a Senior Software Engineer. Below is the list of functional requirements we designed for the Activity Recording module. Analyze them for logical consistency and completeness. Specifically, identify if there are any missing edge cases (e.g., error handling, offline mode) that a standard RASD should include. Do not generate new requirements, but point out the gaps for us to address manually."*

3. **Obtained Outputs**

    - Suggestions for sentence reformulation to make the text more formal and concise.
    - Checklists to verify consistency between different diagrams and textual descriptions.
    - Notifications of potential unclear points in the requirement descriptions.

4. **Verification, Refinement, and Integration** Every output generated by the AI was subjected to a manual critical review by the group members.

    - We did not passively accept the proposed reformulations; instead, we selected and adapted them to ensure they exactly reflected our project specifications.
    - The completeness flags were used as a starting point for internal discussion to manually enrich the document.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 2

Overall Description

2.1 Product Overview

2.1.1 Scenarios

A new user subscribes to the platform

1. An athlete lands on the website's landing page for the first time. After browsing the home screen and viewing the services offered by the app, they decide to sign up.
2. The user clicks the "Sign Up" button, which redirects them to the registration page.
3. At this point, the user must decide whether to register using their own credentials (email and password) or to use an OAuth authentication method, logging in with an account from another service.
4. After filling in the required fields, the user is taken to the second screen of the registration flow, where they are asked to provide additional information needed to calculate performance metrics, such as weight, height, age, etc.
5. Finally, the user is successfully registered and is redirected to the application's main screen.

A user records an activity

1. When a user wants to record an activity, they must go to the "Record Trip" panel.
2. From there, in the "Automatic" section, they simply need to press the play button: the app will start recording GPS data and storing the route and detected potholes.
3. Once the route is completed, pressing the stop button will end the recording.
4. At this point, the user will be asked to verify whether the obstacles detected by the application are actually present or are false positives, and to confirm the condition of the path.
5. The system automatically analyzes the recorded GPS trace to check if it matches an existing public path.
6. If a match is found, the activity is associated with that path, and the validated information is automatically merged with the existing data.
7. If no match is found, a prompt will ask the user whether they want to publish the new route or keep it private.

A user creates a path manually

1. The user logs into the platform using their credentials.
2. Once authenticated, the user accesses the “My Paths” section of their profile.
3. The user selects the “Add Path” option to start creating a new route.
4. The system opens a dedicated page for creating the path.
5. The user enters the required information, including:
    - The name of the path,
    - The positions that compose the path,
    - The condition of the path,
    - The presence of any obstacles or issues present along the route.
6. After completing the input, the user confirms the creation of the path.
7. The system saves the data and displays a confirmation message.
8. The system also offers the user the option to make the path public, so it can be shared with the community.
9. The newly created path is displayed in the “My Paths” section of the user’s profile.

A user searches for a path

1. The user can choose to:
    - log in with their account, or
    - use the platform without authentication.
2. From the home page, the user enters the starting point and the desired destination in the search bar.
3. The system receives the request and queries its database of published paths.
4. The system processes the available options and calculates a score for each path based on:
    - the condition of the segments,
    - the freshness of the information,
    - the effectiveness of the route in connecting the selected points.
5. The system displays the results on the map, ordered in descending order of score.
6. The user reviews the results and can select a specific path to view more details.
7. The system displays detailed information for the selected path, such as:
    - total distance,
    - estimated travel time,
    - overall condition of the route.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

User publishes a path

1. The user accesses the BBP system and opens the “My Paths" section.
2. The system displays the list of paths associated with the user, indicating for each one, among other properties, its status (private or public).
3. The user selects a private route and, using the appropriate button, requests its publication.
4. Upon receiving the publication request, the system asks the user to confirm additional information related to the route:
    - In the case of manually created routes, the user can review and, if necessary, modify the additional information previously entered during creation.
    - In the case of automatically generated routes, the user must confirm or correct the acquired information (if not already done) and may add further details manually.
5. Once the review is complete, the user confirms the publication of the route.
6. The system verifies the data consistency, assigns a unique identifier to the route, and makes it available to the community within the search and browsing functionality.
7. The system notifies the user of the successful publication and updates the route status from private to public.

User modifies the additional information related to a route

1. The user accesses the Best Bike Paths (BBP) platform and opens the “My Paths section.
2. The system displays the list of routes associated with the user, indicating for each one, among other properties, its status (private or public).
3. The user selects a route and opens the section dedicated to additional information.
4. The system displays the current information, including:
    - The condition of the route;
    - The presence of obstacles or damaged sections;
    - The origin of each piece of data (manually entered or automatically acquired).
5. The user can modify, correct, or add new information:
    - If the route was created manually, the user can freely update existing data.
    - If the route was created automatically, the user can review the information acquired by the system and/or add new data manually; in any case, they must confirm or correct the automatically acquired information before publication.
6. The system saves the changes and updates the database.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

2.1.2 Domain Class Diagram

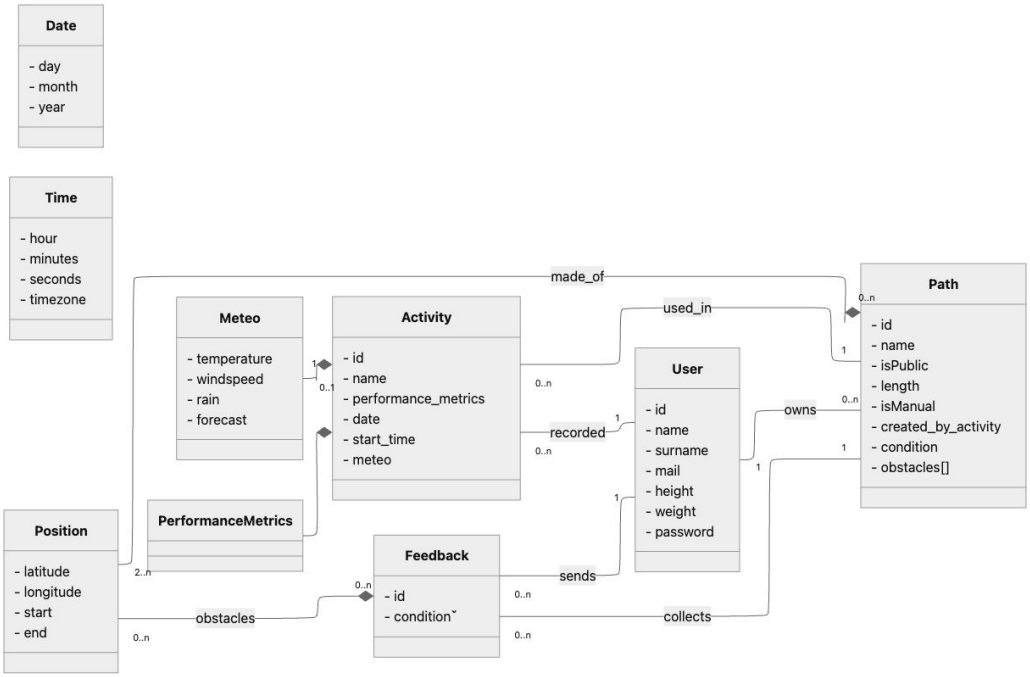


Figure 2.1: Domain Class Diagram

The diagram illustrates the system's static structure:

- **User:** The primary actor of the system. Beyond authentication credentials, this entity stores physical attributes (e.g. height, weight) essential for calculating performance metrics. The User acts as the owner of recorded **Activities**, created **Paths**, and submitted **Feedback**.
- **Path:** The core artifact, defined by metadata (e.g. isPublic) and a geometry composed of **Position** coordinates. It can be created manually or derived from an activity.
- **Activity:** Represents a tracking session with temporal data (e.g. start_time), **PerformanceMetrics**, and **Meteo** conditions. It is linked to the User and optionally to a Path (used_in).
- **Feedback:** Models user reports on path conditions. It references specific **Position** points to spatially locate obstacles (e.g., potholes) along the path's geometry.
- **Position:** The shared geospatial unit ensuring consistency between paths, traces, and feedback locations.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

2.1.3 State Diagrams

Activity Recording

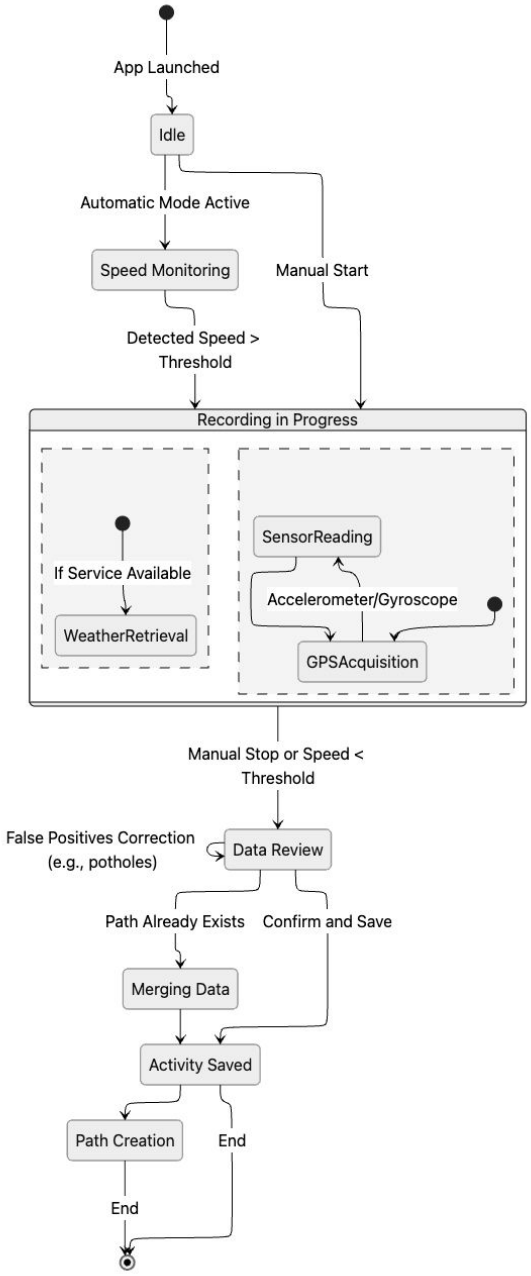


Figure 2.2: Activity Recording State Diagram

This state machine describes the complete lifecycle of a cycling activity recording session.

The process begins in the **Idle** state. From here, the system can transition to the active phase in two distinct ways:

- **Manual Start:** The user explicitly initiates the session, moving directly to **Recording in Progress**.
- **Automatic Detection:** If "Automatic Mode" is active, the system enters the **Speed Monitoring**

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

state. Upon detecting a speed greater than the predefined threshold, it automatically transitions to **Recording in Progress**.

The **Recording in Progress** state is a composite state where the system continuously loops between **GPS Acquisition** and **Sensor Reading** (collecting data from the accelerometer and gyroscope). Concurrently, if the service is available, the system performs a **Weather Retrieval** operation to enrich the activity data.

The recording phase concludes when a "Manual Stop" occurs or the speed drops below the threshold, triggering a transition to the **Data Review** state. This is a mandatory validation phase where the user verifies the collected data and performs **False Positives Correction** (e.g., removing incorrectly detected potholes).

Once the review is confirmed, the system logic branches based on the GPS trace analysis:

1. **Merging Data:** If the system detects that the activity matches an existing public path, it transitions to this state to integrate the new feedback (obstacles/conditions) with the existing path data.
2. **Direct Saving:** If no matching public path is found, the system proceeds directly to **Activity Saved**.

Both flows converge in the **Activity Saved** state. From here, the process either terminates or transitions to **Path Creation**, allowing the user to generate a new path entity from the recorded trace (a process detailed in a separate diagram).

Path Creation

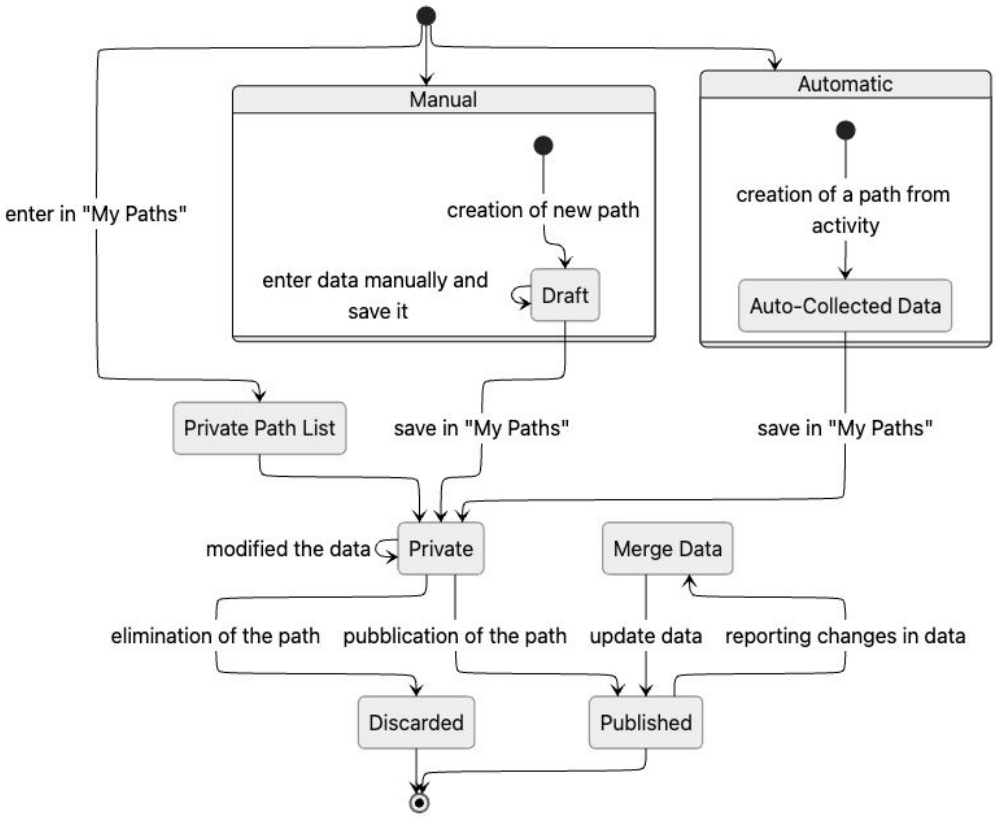


Figure 2.3: Path Creation State Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

The diagram illustrates the creation, management, and evolution process of the Path object within the system. The lifecycle is divided into two macro-phases: instantiation (Manual or Automatic) and state management (Private or Public).

- **Path Instantiation:**
  - **Manual Creation:** The user enters the descriptive data of the path, which is initially managed in a draft state (Draft) within the Manual context. Upon saving, the path transitions to the user's personal list.
  - **Automatic Creation:** The system generates the path based on data collected during a cycling activity (Auto-Collected Data). Once the acquisition is complete, the path is saved and persisted in the private section.
- **Management and Publication:**
  - **Private State:** Once saved (or by accessing "My Paths"), the path resides in the **Private** state. In this phase, the user has full control over the entity: they can modify the data (modified the data cycle), permanently delete the path (transition to Discarded), or proceed to publication.
  - **Published State:** Upon publication, the path becomes visible to the community. In this state, modifications are no longer direct but follow a collaborative process: users can report changes (reporting changes), which are processed by the system via the **Merge Data** function. This function validates and integrates the new information, triggering the actual update of the public data (update data).

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Meteorological Data Retrieval

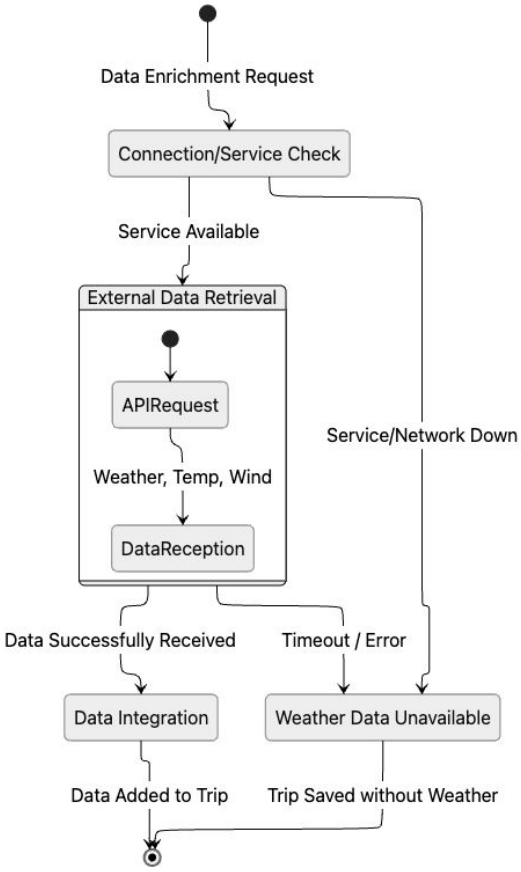


Figure 2.4: Meteorological Data Retrieval State Diagram

This diagram represents the enrichment of an activity with meteorological data. If the device is connected to the network, a request is sent to an external API. If the request is successful, the service retrieves data such as wind speed, temperature, and weather conditions, and formats them in a consistent way. The system then embeds this information into the activity and saves it to the database. If the request fails, the error is handled and the activity is saved without meteorological data.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Path Research

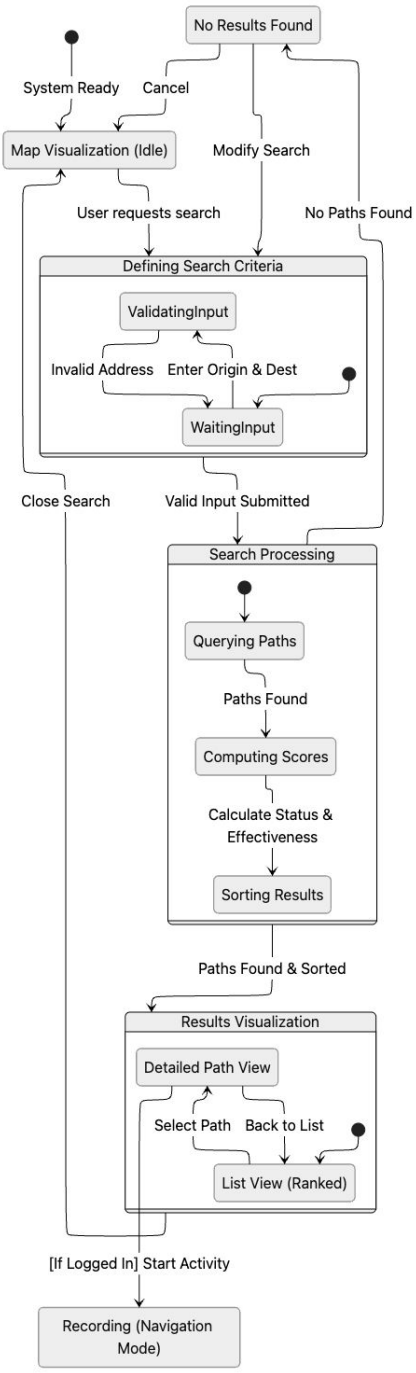


Figure 2.5: Path Research State Diagram

This state machine describes the logic flow of the "Path Search" functionality, accessible to both Guest and Registered users.

The process begins in the **Map Visualization** state. When a user initiates a search, the system transitions to **Defining Search Criteria**, waiting for valid Origin and Destination inputs.

Upon submission, the system enters the **Search Processing** state. This is a critical system-side phase where three operations occur sequentially:

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

1. **Querying Paths:** The system retrieves all geometrically valid paths between the two points.
2. **Computing Scores:** The system calculates a specific score for each path, aggregating its maintenance status and its effectiveness in reaching the destination.
3. **Sorting Results:** Paths are ranked based on the computed score.

If valid paths exist, the system transitions to **Results Visualization**. Here, the user can toggle between a ranked list and a **Detailed Path View** (displaying specific obstacles, weather, and stats).

From the detailed view, a specific transition to **Recording (Navigation Mode)** is available, but guarded by the condition `[If Logged In]`, as only registered users can proceed to track an activity on the selected path.

User Lifecycle

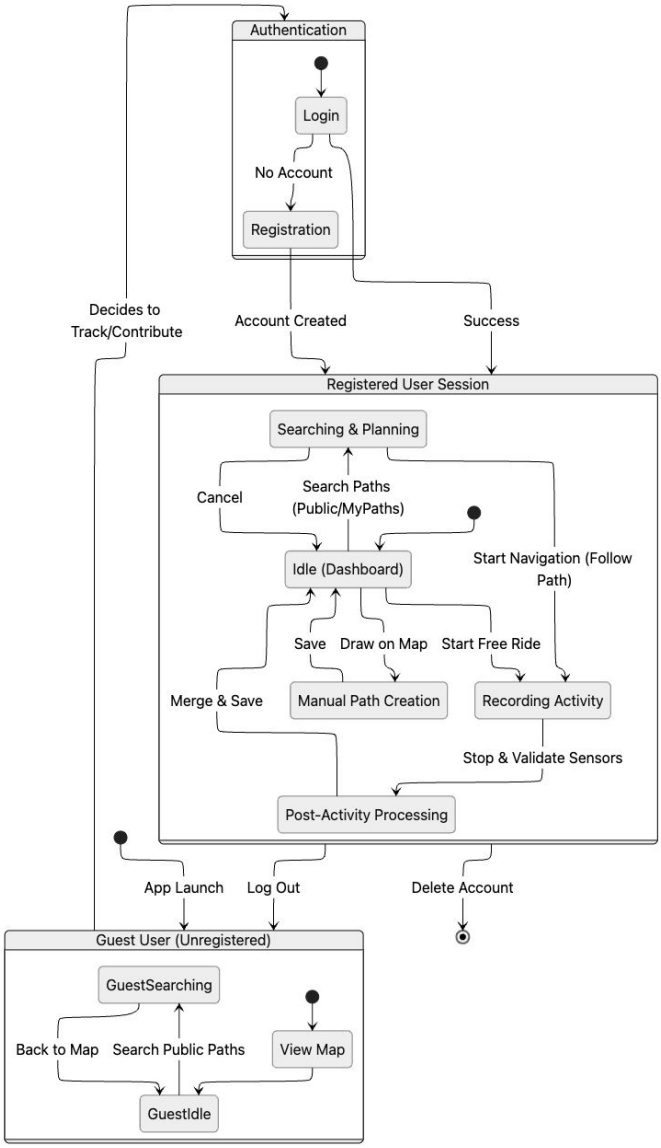


Figure 2.6: User Lifecycle State Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

This state machine diagram describes the user lifecycle within the BBP system, distinguishing between two main macro-states: **Guest User**, unauthenticated, and **Registered User Session**, authenticated.

The lifecycle begins in the **Guest User** state, where functionalities are restricted to read-only operations (represented by the **Guest Searching** state). In this phase, the user can browse the public path inventory but cannot contribute data. To access active functionalities, a transition through the **Authentication** state (Login or Registration) is required.

Once authenticated, the user enters the composite **Registered User Session** state. Here, the **Idle** state acts as a central hub from which three main operational flows branch out:

1. **Searching & Planning:** The user searches for specific paths. From this state, the user can return to the Dashboard or transition directly to **Recording Activity**, following a selected path.
2. **Recording Activity:** This represents the active cycling phase. It can be triggered via the search flow or directly from the Dashboard, not referring to any existing path.
3. **Manual Path Creation:** The user enters an editor mode to manually trace routes on the map without physical recording.

A critical transition occurs at the end of every recording session: the system moves to the **Post-Activity Processing** state. In this mandatory phase, the user validates sensor-detected data and the system executes the merging logic with the existing inventory before returning the user to the **Idle** state. The lifecycle concludes when the user performs a logout or permanently deletes their account.

2.2 Product Functions

Sign-Up and Login

Sign-Up

The system lets the users register in order to manage and record data associated with each of them. When the user accesses the sign-up page for the first time, he's offered the option to log-in with one of their existing accounts through an OAuth service. This way, his BBP account is linked to another account that he owns, making it easier to remember and access for him. The system also gives the user the possibility of entering his email and a new password and to create an account directly inside BBP. In that case, BBP checks that the password is complex enough and that no other account is associated with the same email. If the checks pass, the system stores the information in the DB and grants the user access to the platform.

Login

If the user already has a BBP account or has previously linked their BBP account to one of the other existing accounts that he has through OAuth, the system grants them access the page. If the user access the page through OAuth, a button redirects them to the external account provider's website, which handles the login process and sends the user back only when he completes the login procedure. If, on the other hand, the user registered directly on BBP, the system first checks whether an account associated with the provided email, and then verifies that the password matches. If that's the case, the user is redirected to the website's homepage.

Accessing BBP without accounts

The user can also access BBP without logging in. In this case, the functionalities that are available are more limited. The system lets the user browse between public paths, but all the recording features and the private paths need the login to work

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Activity Recording

The system allows registered users to record their cycling activities. When the user starts the recording, BBP continuously acquires real-time data from the mobile device, including:

- GPS coordinates used to compute distance, average speed, and elevation;
- timestamps associated with all collected events;
- accelerometer and gyroscope data to detect irregularities on the road surface;
- potential stops inferred from speed variations.

During or after the trip, the system processes the collected data and generates a summary reporting distance, duration, average speed, and detected anomalies. If the user decides to save the activity, BBP stores it in the database and makes it available for personal statistics.

Automatic Path Creation

When the user enables the automatic mode, BBP reconstructs the bike path followed during the activity. This includes:

- continuous acquisition of GPS coordinates;
- segmentation of the route based on street structure and direction changes;
- detection of potholes, vibrations, or obstacles through accelerometer and gyroscope data;
- estimation of the user's speed to determine whether they are biking.

At the end of the activity, BBP generates a draft path containing:

- a map of the complete route;
- a list of detected street segments;
- automatically identified road conditions and anomalies.

Since the automatically detected data may contain false positives, the user must review, confirm, or correct all generated information before it becomes publishable. After confirmation, the path and related annotations are stored and made available to the community.

Meteorological Data Integration

When available, BBP enriches recorded activities with meteorological information obtained from an external weather service. After an activity is completed, the system sends:

- the activity timestamps;
- the geographical coordinates of the route.

The external service returns weather conditions for the corresponding time and location, including:

- temperature;
- general weather conditions (e.g., sunny, rainy, cloudy);
- wind speed and direction;
- possible weather alerts.

BBP associates the retrieved meteorological data with the recorded activity before storing it. These details are shown in the activity summary and are available for statistical analysis.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Manual Path Creation

In addition to the automatic creation of bike paths, registered users can also create paths manually by directly entering the required data. During this process, the platform asks the user to specify the name of the path, the streets that compose it, the condition of each segment, and the possible presence of obstacles or issues. Once the information has been confirmed, the system stores the data in its database and allows the user to decide whether to make the path publishable, meaning visible to the entire community. If the path is not made publishable, it remains visible only to the user who created it, within their personal profile.

Path Search by Origin and Destination

Any user, including unregistered ones, can search for a bike path between two points using the search algorithm provided by the platform. The system queries its database of publishable paths and identifies those that connect the specified origin and destination points. The results are displayed on an interactive map and sorted in descending order based on a path score automatically computed by the platform. The user can select one of the displayed paths to view detailed information, such as the total distance, estimated travel time, and overall path conditions.

Path Ranking

The score assigned to each path is determined by considering several factors:

- Path condition, where better-maintained paths receive higher scores;
- Freshness of information, favoring paths with more recent data;
- Path effectiveness, meaning how efficiently the path connects the two selected points.

Entering route condition information

Each route, whether derived from an automatically recorded activity or manually created by the user, is characterized by a set of additional information describing its condition and rideability. In particular, this information concerns:

- The condition of the roads that make up the route;
- The presence of obstacles, such as potholes, surface damage, or dangerous sections.

Additional information is always managed through the same interface, but with different behavior depending on how the route was created:

- **Manual mode** During route creation, the user directly enters information about the condition of the roads and any obstacles.
- **Automatic mode** The system acquires data via the device's sensors (GPS, accelerometer, gyroscope); therefore, the user finds this information already filled in. Each piece of information must be confirmed.

Regardless of the mode, the user can always modify or update the additional information before and after publication, thus contributing to improving the quality of shared data.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Paths publication

Registered users can choose to publish a route, making it visible and accessible to other users within the platform. In particular, publication adds the route to *BBP*'s public archive, making it available to all other users, including non-registered ones, through the route search and browsing functionality. Only published routes can be viewed by others and contribute to enhancing the quality of shared community data. Publication always concerns the complete route, including all additional information such as road conditions and the presence of any obstacles.

Merging information from different users

When multiple users provide information regarding the same route or overlapping sections, the BBP system automatically merges the received data concerning road conditions and the presence of obstacles. The merging process takes into account two main factors:

- The freshness of the information, meaning how recently it was provided;
- The number of confirmations received from different users for the same condition or status.

In the case of conflicting reports, the system applies a weighted evaluation mechanism that prioritizes the most recent and most frequently confirmed information.

2.3 User Characteristics

2.3.1 Registered User

A registered user has a personal profile containing the information provided during registration, the saved paths, and the cycling activities they have performed. They can save their rides, view detailed statistics (such as distance, average speed, and total time), and contribute to updating the path inventory by reporting the condition of the routes or the presence of obstacles. The registered user can also create new paths, either manually by entering data directly or automatically during an activity, using the sensors of their mobile device. After creation, the user can make these paths publishable, allowing the community to access and view them. Finally, the user can search for bike paths between two points and view them on the map, ordered according to the score assigned by the system.

2.3.2 Unregistered User

An unregistered user can access a limited version of the application, which only allows the consultation of publishable paths available on the platform. They can search for paths between two points, display them on the map, and view basic information such as the name, score, and general condition of each path, but they cannot perform any additional actions on them.

2.4 Assumptions, dependencies and constraints

2.4.1 Assumptions

A1. **Path Optimality Definition**: A path is considered "better" than another if it minimizes the travel distance or if its start/end points are closer to the user's desired origin/destination.

A2. **Validation Constraint**: While user confirmation can happen anytime for private activities, automatically detected data (sensors) must be validated by the user before being used for public updates.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

A3. **Recording Implies Path Data**: It is assumed that recording a sporting activity implicitly involves collecting path data (GPS/Status) for potential path creation.

A4. **Contribution via Merging**: Once a path is public, its core geometry cannot be overwritten; updates happen through feedback/reports that are merged by the system.

A5. **Manual Creator Competence**: It is assumed that when a user creates a path manually, they have physically traversed it and possess knowledge of its current status.

A6. **Path Uniqueness**: The system assumes that no two identical public paths (same geometry) can exist in the database.

A7. **Manual Duplicate Avoidance**: Automatic duplicate detection relies on GPS tracks. For manual creation, it is assumed the user checks the map for duplicates before creating a new path.

2.4.2 Dependencies

D1. **External weather service**: The system relies on the availability and accuracy of GPS data from users' devices to record cycling activities and create paths.

D2. **Mapping service**: Offers geographic data and map layers used to visualize routes, match GPS tracks to road segments and retrieve details about paths, intersections, and terrains.

D3. **Storage database**: Stores all platform data, including user routes, additional information (road conditions, obstacles), user profiles and publication metadata. Ensures persistence, consistency, and efficient retrieval.

D4. **Authentication services**: Manages user identity verification, login processes and secure access to personal data and features.

D5. **Icon service**: Provides a library of graphical icons used across the interface for consistent and intuitive representation of road conditions, obstacles, route statuses, and other UI elements.

2.4.3 Constraints

C1. **Privacy regulations**: The system must comply with relevant data protection laws and regulations (e.g., GDPR) to ensure user data privacy and security.

C2. **Automatic recording Accuracy limits**: The automatic activity recording feature will operate within the technical accuracy limits of mobile device sensors (GPS, accelerometer, gyroscope). The system must account for possible measurement noise, positioning errors and sensor variability across devices.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 3

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Below are the main screenshots of the application. The first ones show the features available to both logged-in users and anonymous visitors, such as the homepage and the public path search.

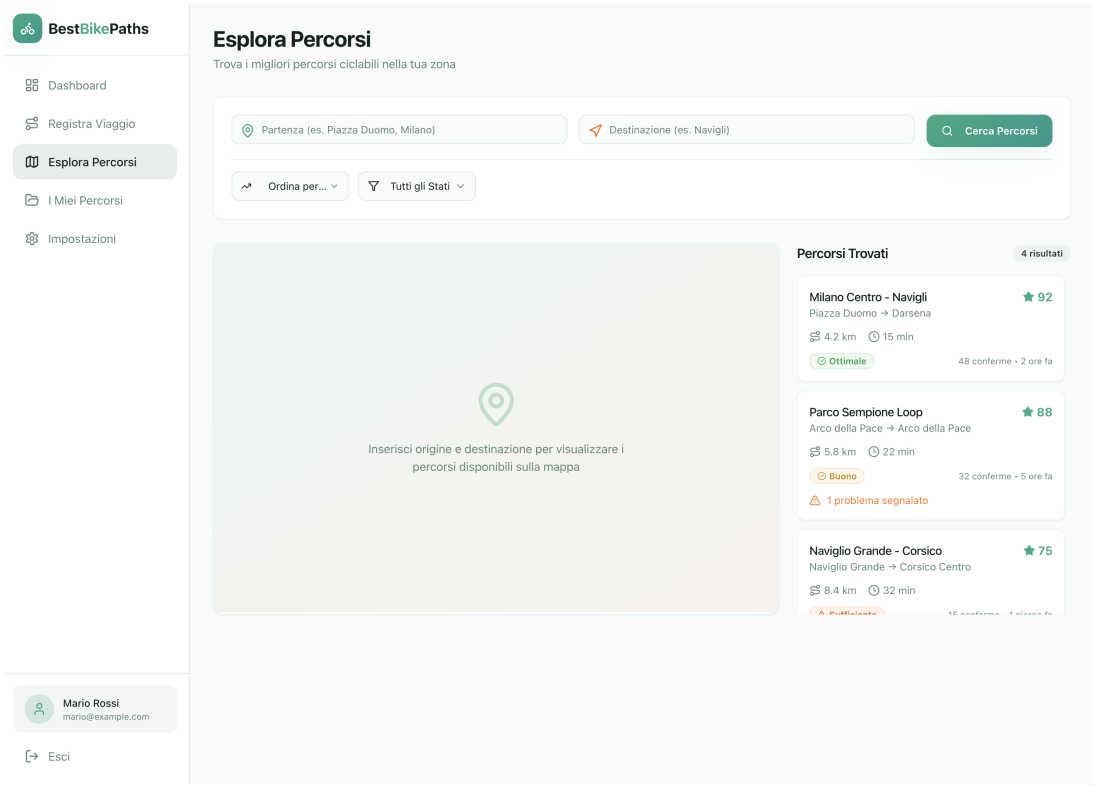


Figure 3.1: Public path search

The following screens are accessible only after authentication: the profile page, the activity recording interface, and the section where users can view their personal paths, both public and private.

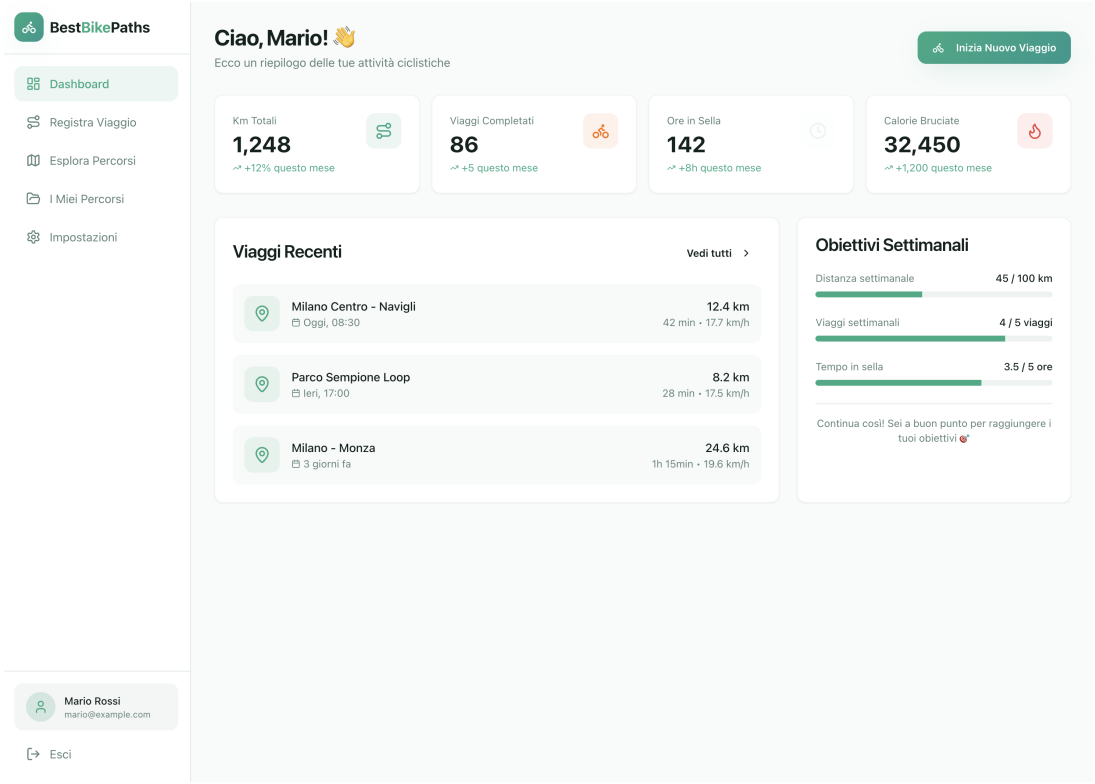


Figure 3.2: Dashboard

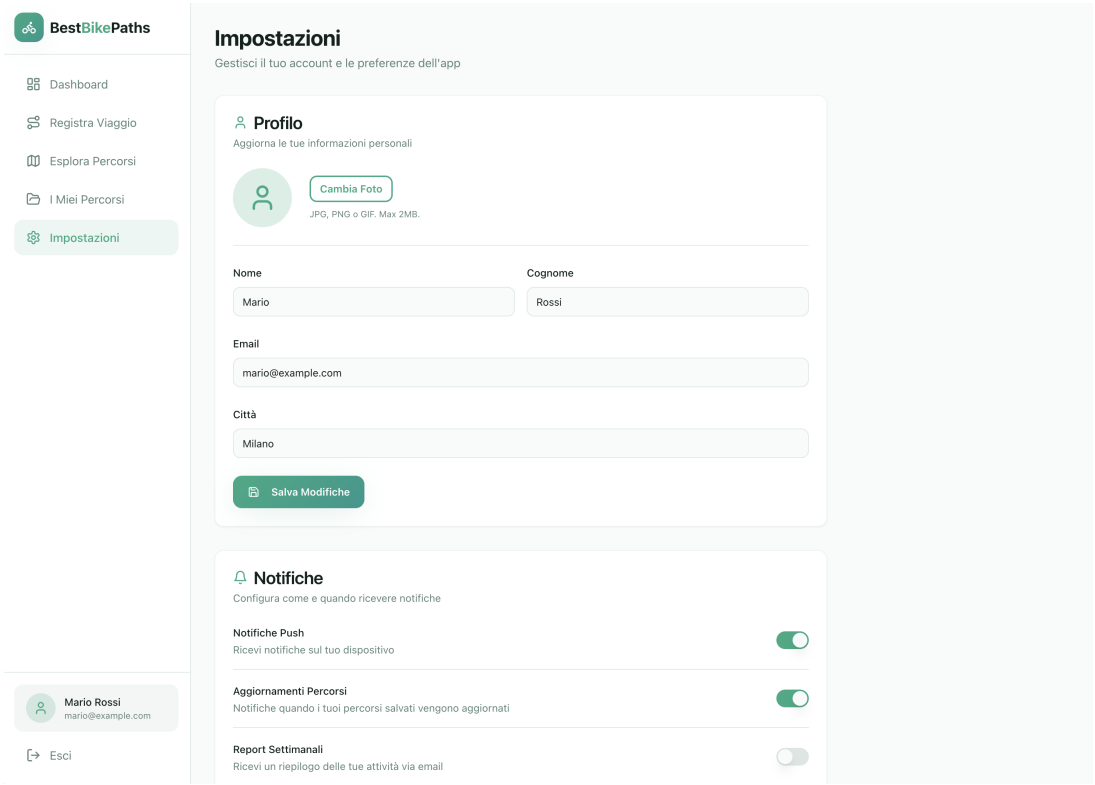


Figure 3.3: Profile page

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

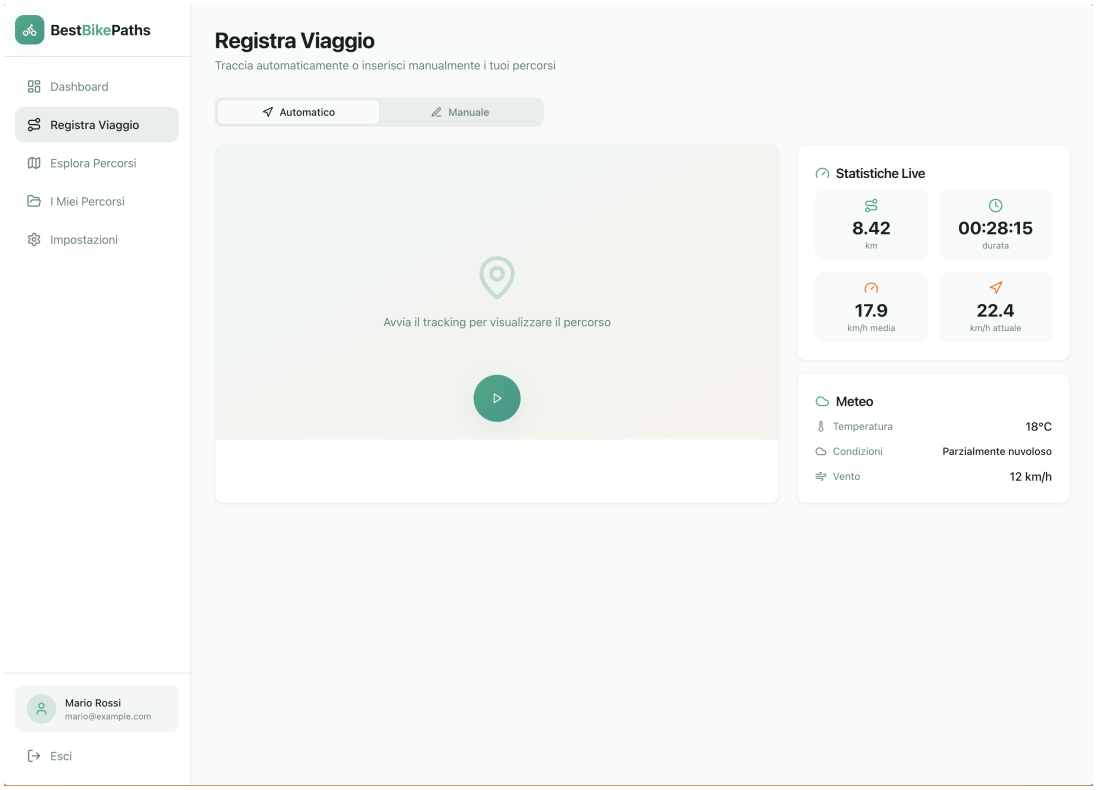


Figure 3.4: Activity recording interface

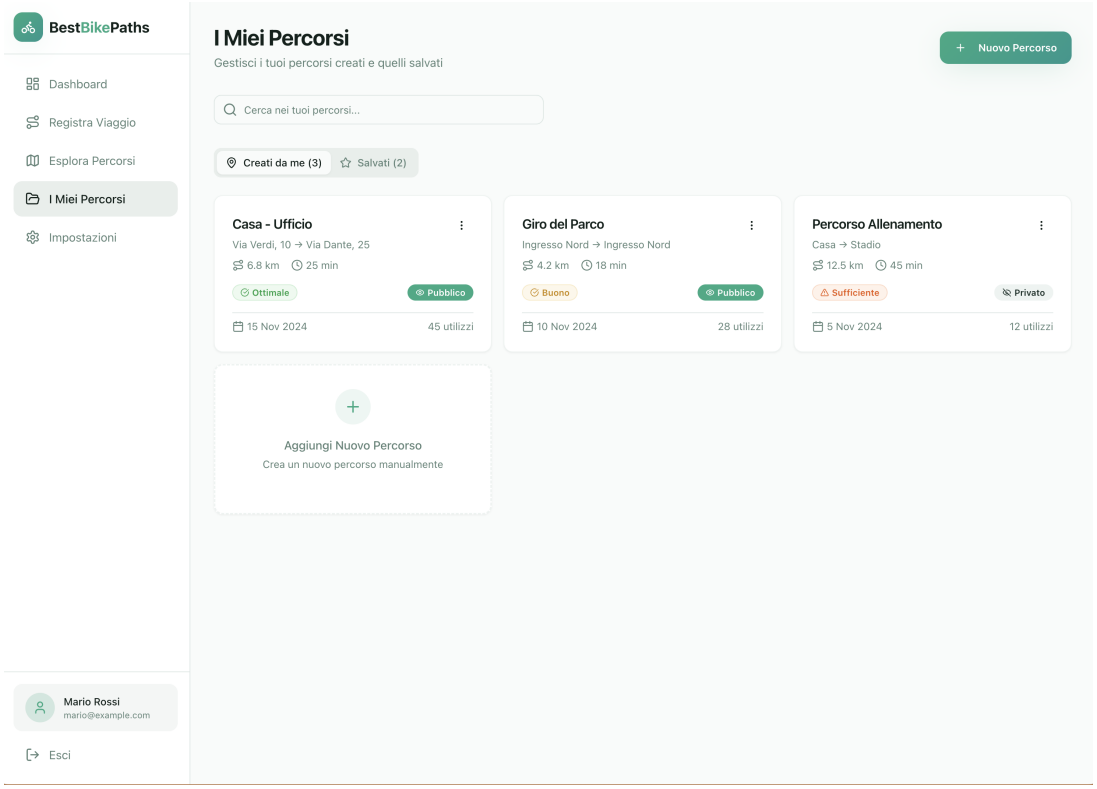


Figure 3.5: User's personal paths

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3.1.2 Hardware Interfaces

Since this is a web application, no additional hardware is required beyond a device capable of connecting to the internet. This applies to path browsing and manual activity creation.

However, to use the automatic recording feature, the device must include specific sensors—namely an accelerometer, GPS, and a compass.

3.1.3 Software Interfaces

- **Database:** To store user credentials, paths, and activities, the system must rely on a dedicated database accessible by the platform.
- **Identity provider:** The system may allow users to register with an email and password or authenticate through an external OAuth provider. This lets users access the platform without creating new credentials, using an account they already have with another service. In this case, an interface to communicate with the chosen provider is required to validate the user's identity.
- **Meteorological data provider:** When an automatic activity is recorded, the user can enrich it with weather information retrieved from an external API. To enable this, the system must expose a secure endpoint capable of requesting and receiving accurate meteorological data.

3.1.4 Communication Interfaces

The Best Bike Paths (BBP) system is a distributed application primarily relying on a Serverless architecture (e.g., using Function as a Service, FaaS) to manage backend processing. Communication interfaces must securely handle the transmission of sensitive data (such as user location) and high volumes of telemetric data.

- **User and Client-Function Communication:** All interactions between clients (web browser interfaces and mobile devices) and the backend compute layer (serverless functions) must utilize HTTPS to ensure end-to-end encryption and data security.
- **Mobile Sensor Data Transmission:** The mobile application is responsible for securely transmitting data streams from device sensors (GPS, accelerometer, and gyroscope) directly to the database. These inputs are critical for the automatic trip tracking mode and the detection of road defects (potholes). The transmission mechanism must be optimized to minimize bandwidth consumption and battery drain, utilizing efficient, asynchronous upload of collected data.
- **External Interfaces (Meteorological Service):** The BBP system interacts with an external third-party meteorological service via its public APIs (a Machine-to-Machine interface). This external interface is used by the backend functions to retrieve real-time weather data (conditions, temperature, wind speed) to enrich the saved trip data.
- **Authentication and Authorization:** User access is managed via secure mechanisms (e.g., using OAuth). These mechanisms facilitate access control to the serverless functions that manage data storage and processing, ensuring only authenticated users can access their personalized features.

3.2 Functional Requirements

Sign Up & Login

The platform allows users to create an account, access the system, and recover credentials. Guest access is limited.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

- **R1** The platform allows users to create an account by providing the required personal information.
- **R2** The platform allows registered users to log in and access the system.
- **R3** The platform allows registered users to recover the access credentials.
- **R4** The platform allows guest users (non-registered users) to access only the path search functionality of the system.

Activity recording

The system enables users to track their cycling activities and collect sensor data.

- **R5** The system shall allow registered users to start recording a new cycling activity.
- **R6** The system shall allow users to select an existing path from the path public repository to follow during the activity (not mandatory).
- **R7** The system shall collect raw data from the mobile device's sensors (GPS, accelerometer, gyroscope) during the recording of an activity.
- **R8** The system shall analyze the user's speed derived from GPS data to verify if the user is effectively cycling.
- **R9** The system shall analyze accelerometer and gyroscope data to detect potential obstacles or road surface anomalies.
- **R10** The system shall calculate real-time statistics during the recording, including total distance covered and average speed.
- **R11** The system shall allow the user to pause and resume the recording of an activity.

Path creation

The system handles the creation of new paths through two modes: automatic detection from an activity or manual creation via map.

Automatic Creation

- **R12** Upon stopping an activity, the system shall display the list of automatically detected obstacles.
- **R13** The system shall require the user to validate the detected obstacles before proceeding.
- **R14** After validation, the system shall analyze the recorded GPS trace to identify if it matches an existing path in the database.
- **R15** If the recorded trace matches an existing public path, the system shall automatically associate the activity with that path entity.
- **R16** The system shall use the validated data to update the status and information of the existing public path.
- **R17** The system shall add the identified path to the "My paths" section if not already present.
- **R18** If the recorded trace doesn't match any existing public path, the system shall allow users to generate a new path entity starting from the GPS trace of a recorded activity.
- **R19** The system shall allow the user to associate a name to the recorded path and store it in the "My paths" section.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Manual Creation

- **R20** The system shall allow the user to create a new path manually by selecting a sequence of streets from a map.
- **R21** The system shall require the user to manually input the status of the streets composing the path and mark the position and type of obstacles along it.
- **R22** The system shall allow the user to assign a name to the manually created path and store it in the "My Paths" section.

Path search from origin to destination

- **R23** The platform allows users to search for cycling paths based on a starting point and an ending point.
- **R24** The platform allows users to filter the search results based on the path parameters.

Path ranking

- **R25** The platform is able to rank the available paths based on the path data, including the condition of each segment, the freshness of the information, and the effectiveness of the path in connecting the selected points.

Entering path condition information

- **R26** The platform is able to detect when a registered user follows a path that is already available to the public and associate the activity with that path.
- **R27** The platform allows registered users to update the condition information of a path, even without performing an activity on that path.

Path management and publication

- **R28** The system shall allow users to edit the information (name, street status, obstacles) of any path they have created.
- **R29** The system shall allow users to publish a stored path, making it available to the community.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3.2.1 Use case diagrams

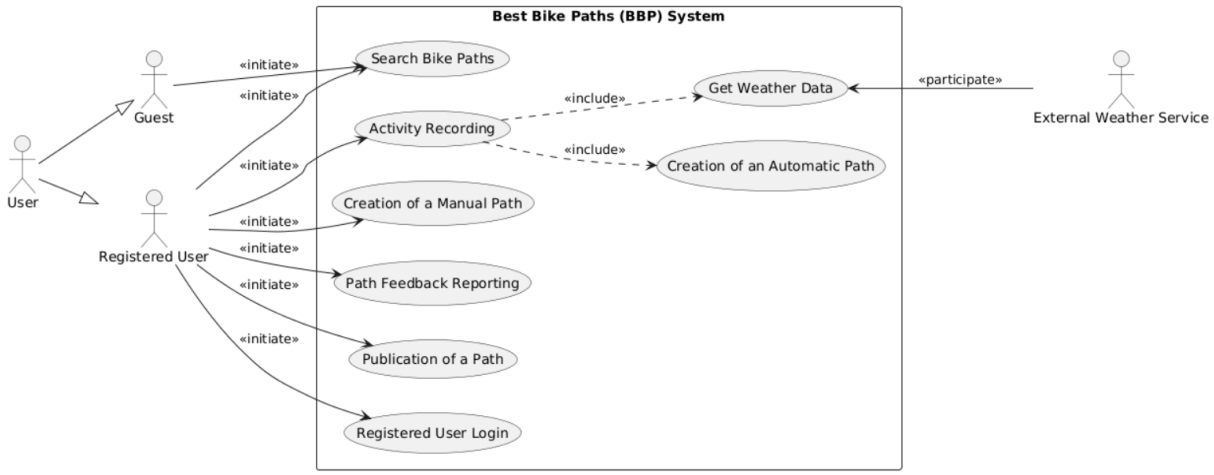


Figure 3.6: Utilities

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3.2.2 Use cases

[UC1] User Registration

Name	**User Registration**
Actors	User, BBP App, Auth service
Entry Condition	User opens BBP application without being authenticated
Event Flow	(a) The system displays the welcome screen with options: “Log In”, “Sign Up”, “Continue as Guest” (b) The user selects “Sign Up” (c) The system shows two options: Sign up with an external OAuth provider or create an account with email and password Main Flow (Standard Email Registration) : (d1) The user chooses email registration (e1) The user enters an email and a password (f1) The system verifies that the password is complex enough and that no existing account is associated with the given email (g1) The system stores the new user information in the DB (h1) The system grants the user access to the platform as a registered user Alternative Flow (OAuth Registration): (d2) The user selects an external authentication provider (e2) The Authentication Service validates the user’s identity (f2) The system creates a new account linked to the OAuth identity (g2) The system grants the user access to the platform as a registered user
Exit Condition	The user accesses the system with a Registered Session
Exception	Password Too Simple: the system shows an error message and asks the user to enter a stronger password. Email Already Used: the system alerts the user and suggests entering a different email or logging in with the existing account. OAuth Authentication Failed: the system displays an error and returns to the authentication options.

Table 3.2: Use Case: [User Registration]

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC2] System Access

Name	**System Access**
Actors	User, BBP Platform, Auth service
Entry Condition	User opens BBP application without being authenticated
Event Flow	(a) The system displays the welcome screen with options: “Log In”, “Sign Up”, “Continue as Guest” Main Flow (Login): (b1) The user selects “Log In” (c1) The user enters email and password (d1) The system validates credentials against the db (e1) The system retrieves the user profile and redirects to the User Dashboard Alternative flow (Guest): (b2) The user selects “Continue as Guest” (c2) The system enables “Limited Mode” (Search only, no Recording) (d2) The system redirects to the Search View.
Exit Condition	The user accesses the system with a Registered Session or a Guest Session
Exception	Invalid Credentials: the system displays an error and asks to retry

Table 3.4: Use Case: [System Access]

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC3] Activity Recording

Name	**Activity recording**
Actors	Registered user, BBP Platform, Device sensors, External Weather Service
Entry Condition	User is logged in and GPS is enabled on the device
Event Flow	(a) The user presses “Start Recording” (b) The system starts collecting data from GPS, Accelerometer and Gyroscope (c) The system calculates real-time statistics and detects anomalies (d) The user presses “Stop activity” and the system pauses data collection (e) The user presses “Save activity” (or decides to resume) (f) The system presents the list of automatically detected obstacles for validation (g) The user confirms or modifies the detected obstacles (h) The system retrieves meteorological data for the recorded area/time from the External Service (i) The system checks for matching existing paths and saves the activity, merging data or creating a new path entry
Exit Condition	The activity is saved in “My Paths”
Exception	GPS signal is lost: system pauses the recording and notifies the user Weather service is unavailable: data is saved without weather information

Table 3.6: Use Case: [Activity Recording]

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC4] Creation of a Manual Path

Name	**Creation of a Manual Path**
Actors	User, BBP App, BBP Server
Entry Condition	The user is authenticated and navigates to the “My Paths” section
Event Flow	(a) The user selects “Create New Path” (b) The platform presents a form requesting all the necessary information of the new path (c) The user enters the data and submit the form (d) The BBP server validates the submitted data (e) The system checks whether a similar path already exists in the public database (f) The platform create a new private path associated with the user (g) The BBP platform confirms creation and displays the new path in the “My Paths” section
Exit Condition	A new private path is successfully created and visible within the user’s “My Paths” section
Exception	Duplicate Public Path Found: the system informs the user that a very similar path already exists in the public database and suggests viewing the public version instead. The new path is not created Invalid or Incomplete Data: the system displays an error and asks the user to correct the form.

Table 3.8: Use Case: [Creation of a Manual Path]

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC5] Publication of a Path

Name	**Publication of a Path**
Actors	User, BBP App, BBP Server
Entry Condition	The user is logged in and navigates to the "My Paths" section. The user has at least one private
Event Flow	(a) The user selects a specific path to view its details (b) The system shows the path information and available actions, including "Publish Path" (c) The user clicks on the "Publish Path" (d) The Platform validates the request (e) The platform makes the path public and visible to all the users
Exit Condition	The path is stored in the system with status Public, and it becomes visible to all platform users
Exception	Path Already Public: the system informs the user that the selected path is already public Unauthorized User: the user attempts to publish a path they do not own and the system denies the action

Table 3.10: Use Case: [Publication of a Path]

[UC6] Path Feedback reporting

Name	**Path Feedback reporting**
Actors	Registered User, BBP Platform
Entry Condition	The user is viewing the details of a path, from "My paths" or from the public repository
Event Flow	(a) The user selects "Report issue" or "Update Status" button on the path detail screen (b) The system displays a form overlaid on the path map (c) The user selects the type of report and pinpoints the location along the path (d) The user inserts the details and confirms (e) The system validates the location consistency and executes the internal merging algorithm (f) The system updates the db and confirms submission to the user
Exit Condition	The user feedback is successfully merged into the system
Exception	Location Mismatch: The user attempts to place a feedback marker too far from the actual path geometry. The system rejects the input.

Table 3.12: Use Case: [Path Feedback reporting]

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC7] Search for a Path

Name	**Search for a Path**
Actors	User, BBP App, BBP Server
Entry Condition	The user is in the "Search" section of the application (authentication not required)
Event Flow	(a) The user enters the starting and the destination point of the path, that he wants to search (b) The user may optionally apply filters (c) The system validates the user inputs (d) The platform queries the database for all public paths that satisfy the user's search parameters and the applied filters (f) The system computes the path score for each candidate route based on: path condition, freshness of a data and effectiveness in connecting origin and destination (g) The system orders the results in descending order according to the score (h) The platform displays the matching paths on an interactive map and provides a list view (i) The user may select a specific path to see its details
Exit Condition	The BBP platform displays all the search results, sorted by path score, and shows them on the map
Exception	No Result Found: the system displays the message: "No public paths match your search criteria."

Table 3.14: Use Case: [Search for a Path]

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3.2.3 Sequence diagrams

[UC1] User Registration Sequence Diagram

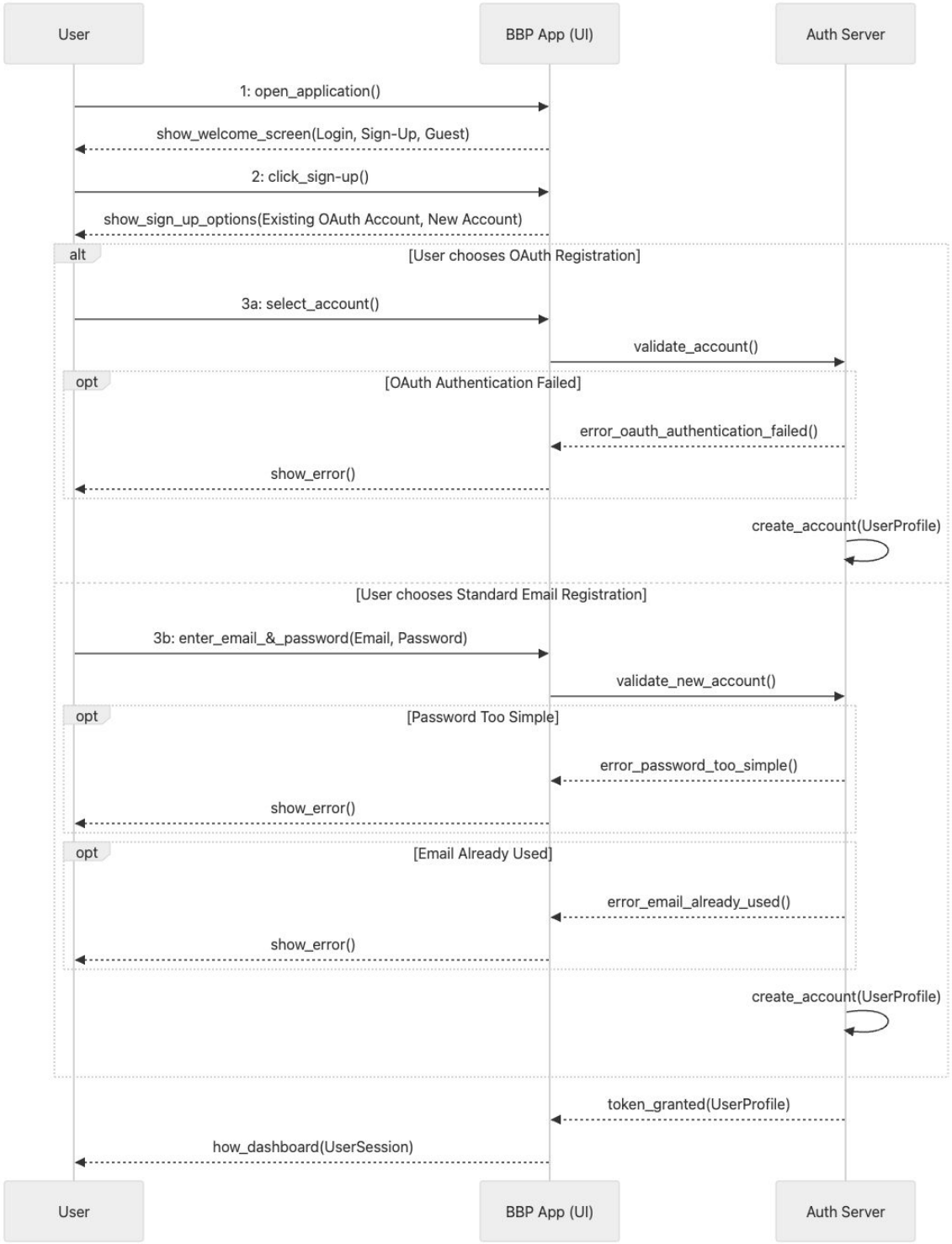


Figure 3.7: [UC1] User Registration Sequence Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC2] System Access Sequence Diagram

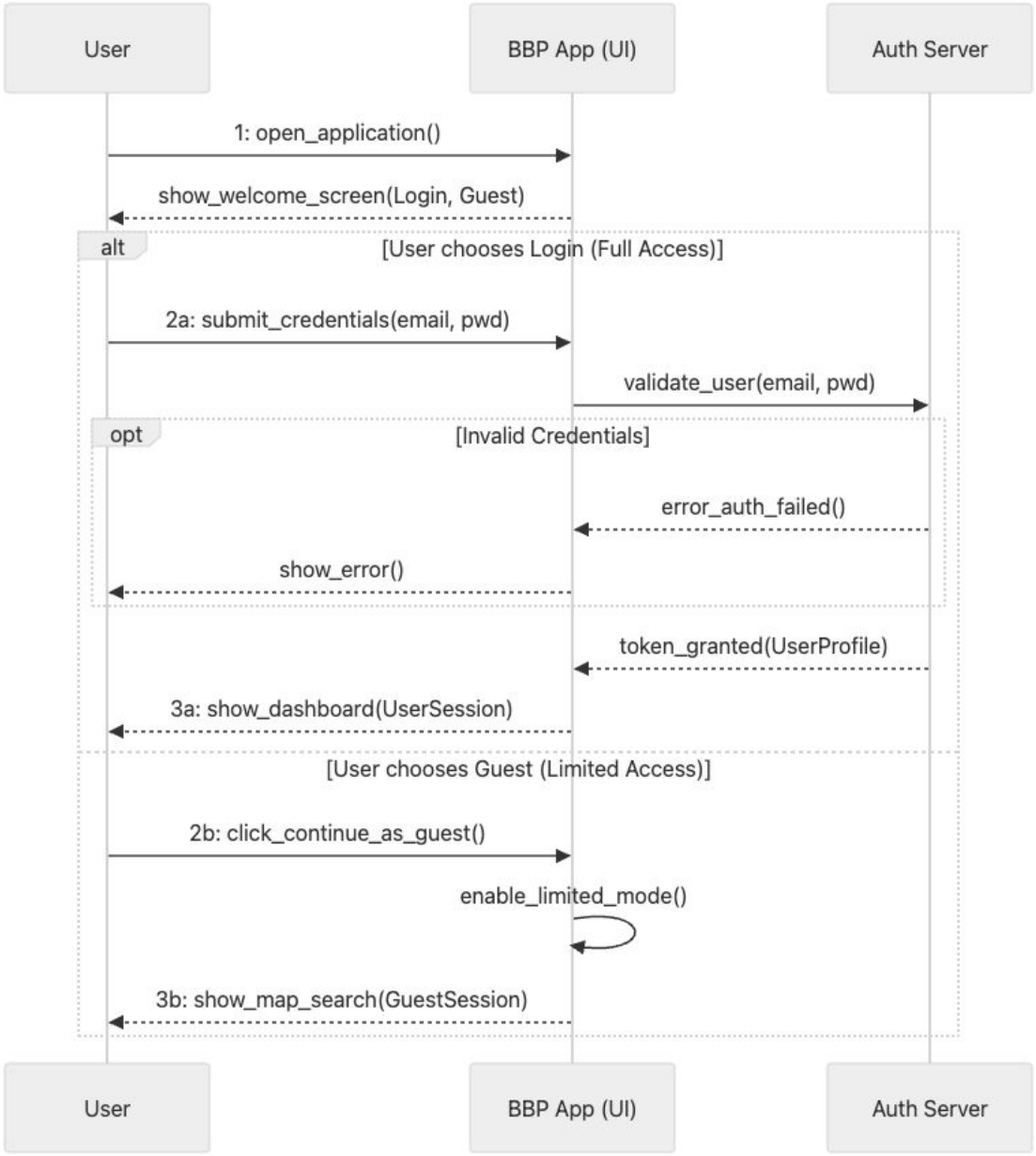


Figure 3.8: [UC2] System Access Sequence Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC3] Activity Recording Sequence Diagram

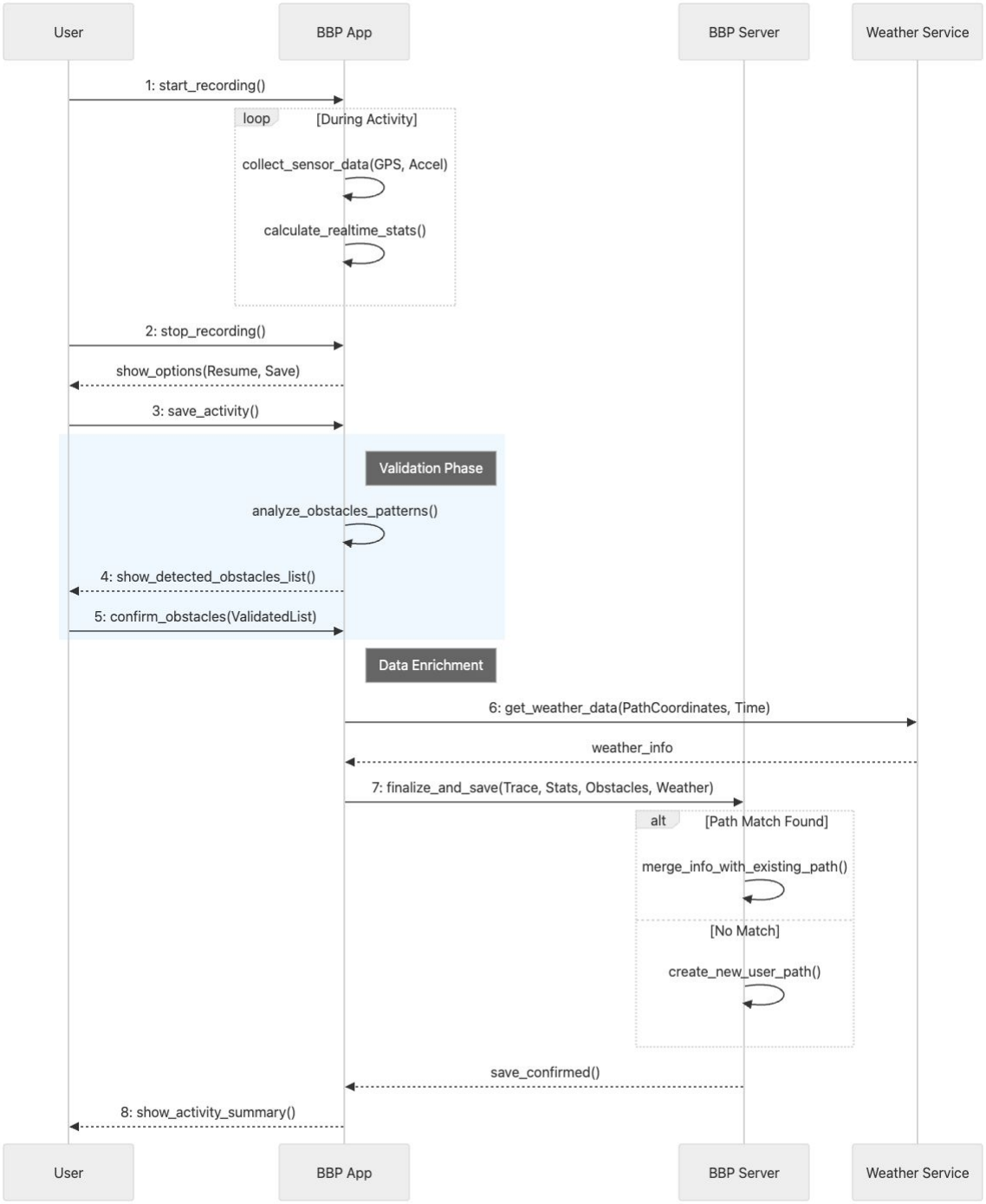


Figure 3.9: [UC3] Activity Recording Sequence Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC4] Creation of a manual path Sequence Diagram

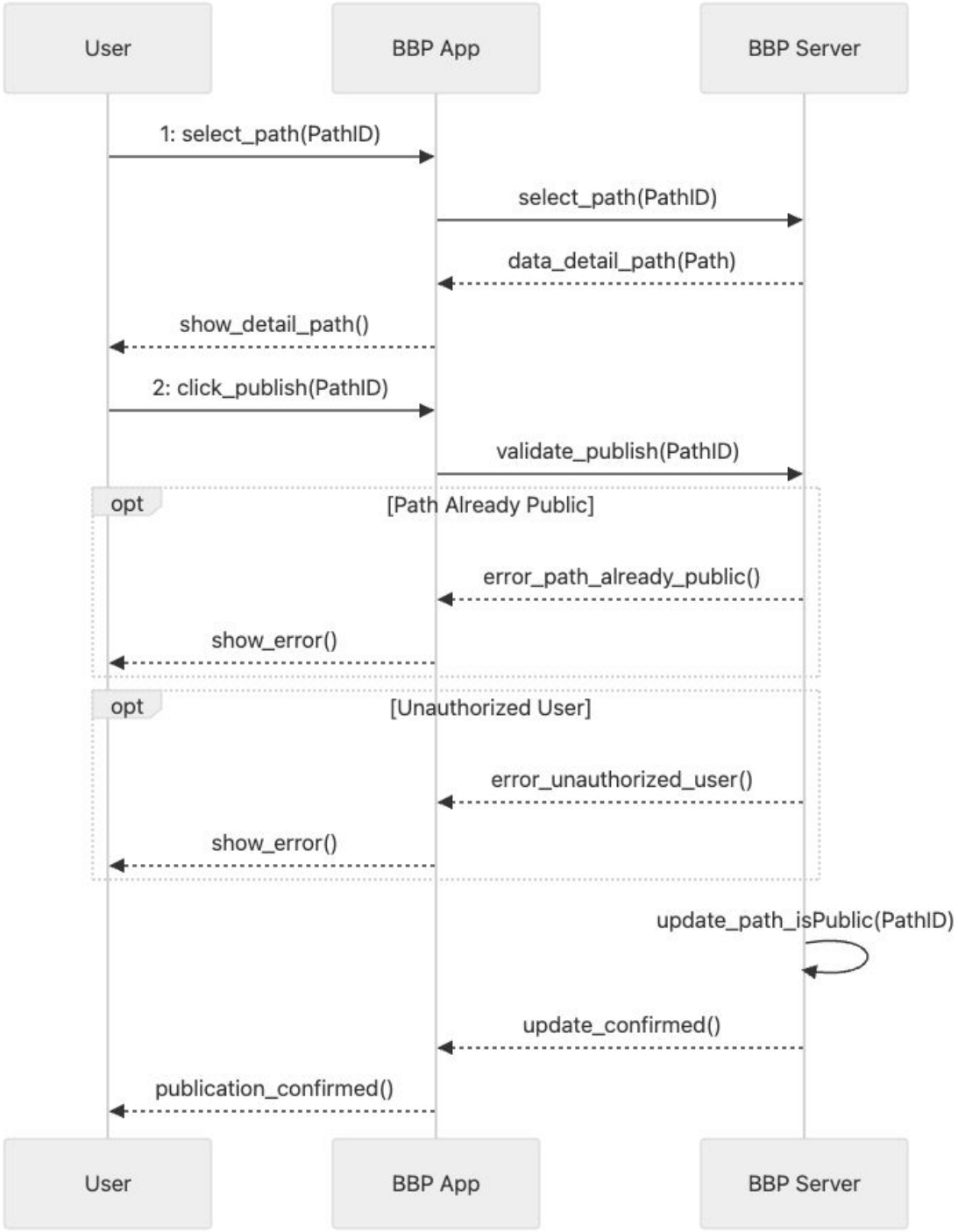


Figure 3.10: [UC4] Creation of a manual path Sequence Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC5] Publication of a path Sequence Diagram

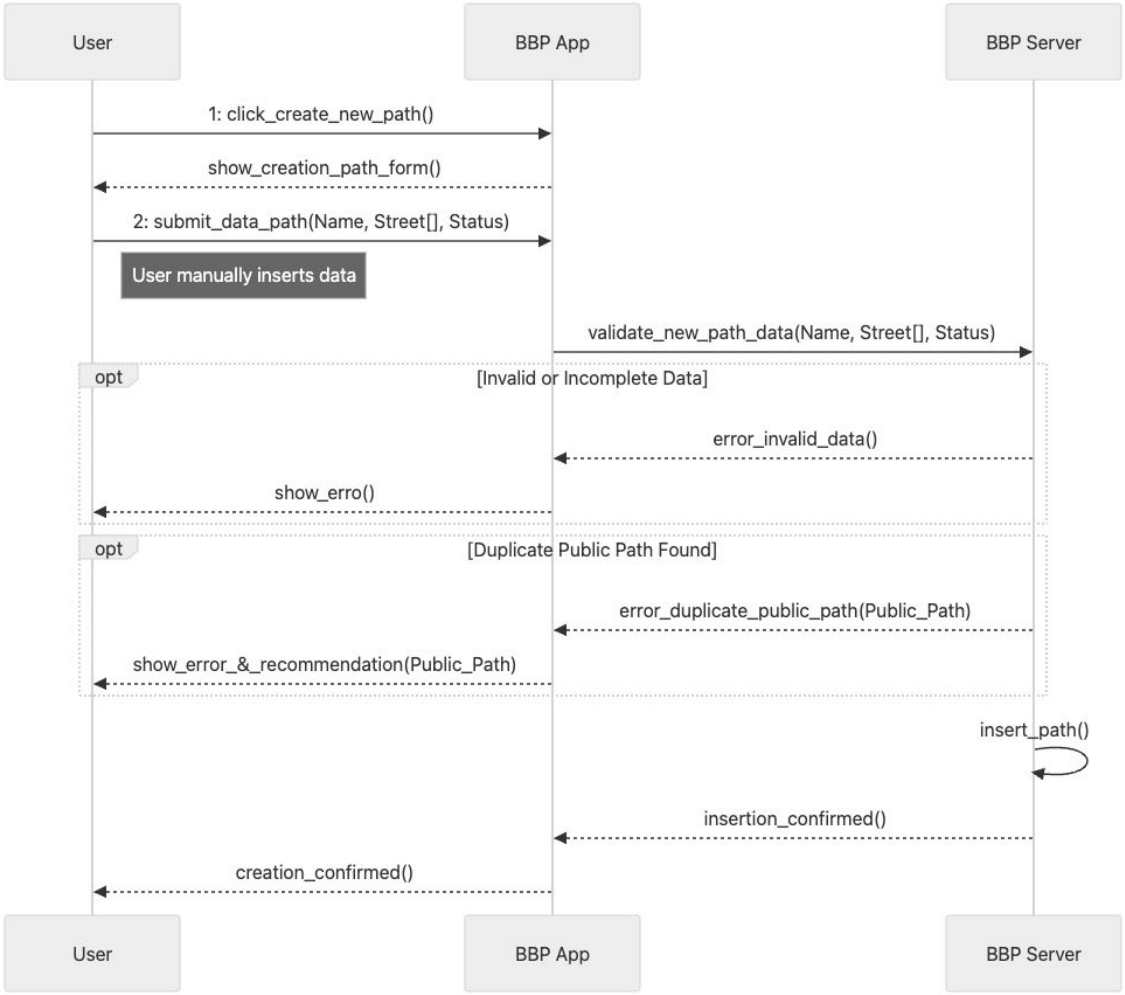


Figure 3.11: [UC5] Publication of a path Sequence Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC6] Path Feedback reporting Sequence Diagram

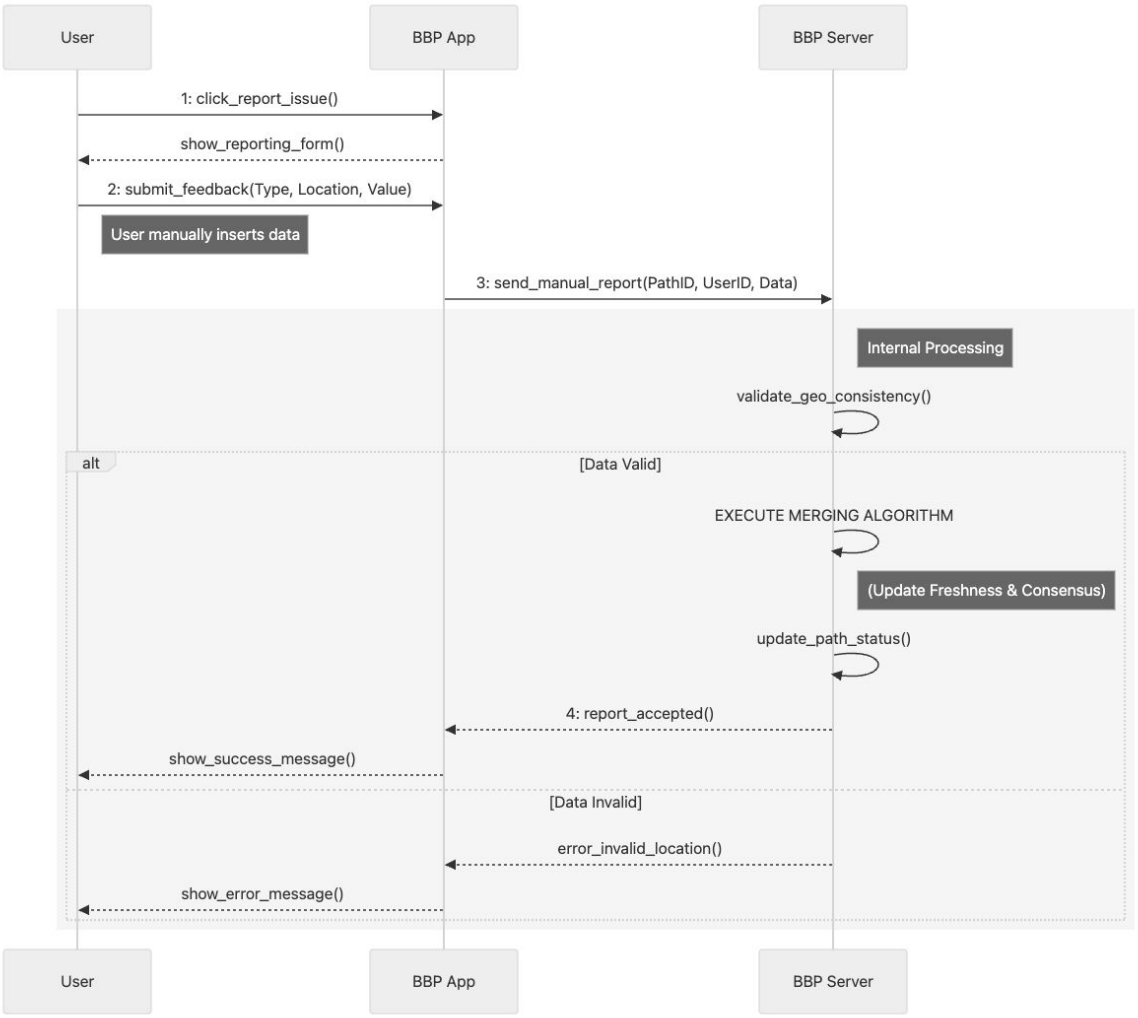


Figure 3.12: [UC6] Path Feedback reporting Sequence Diagram

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

[UC7] Search for a path Sequence Diagram

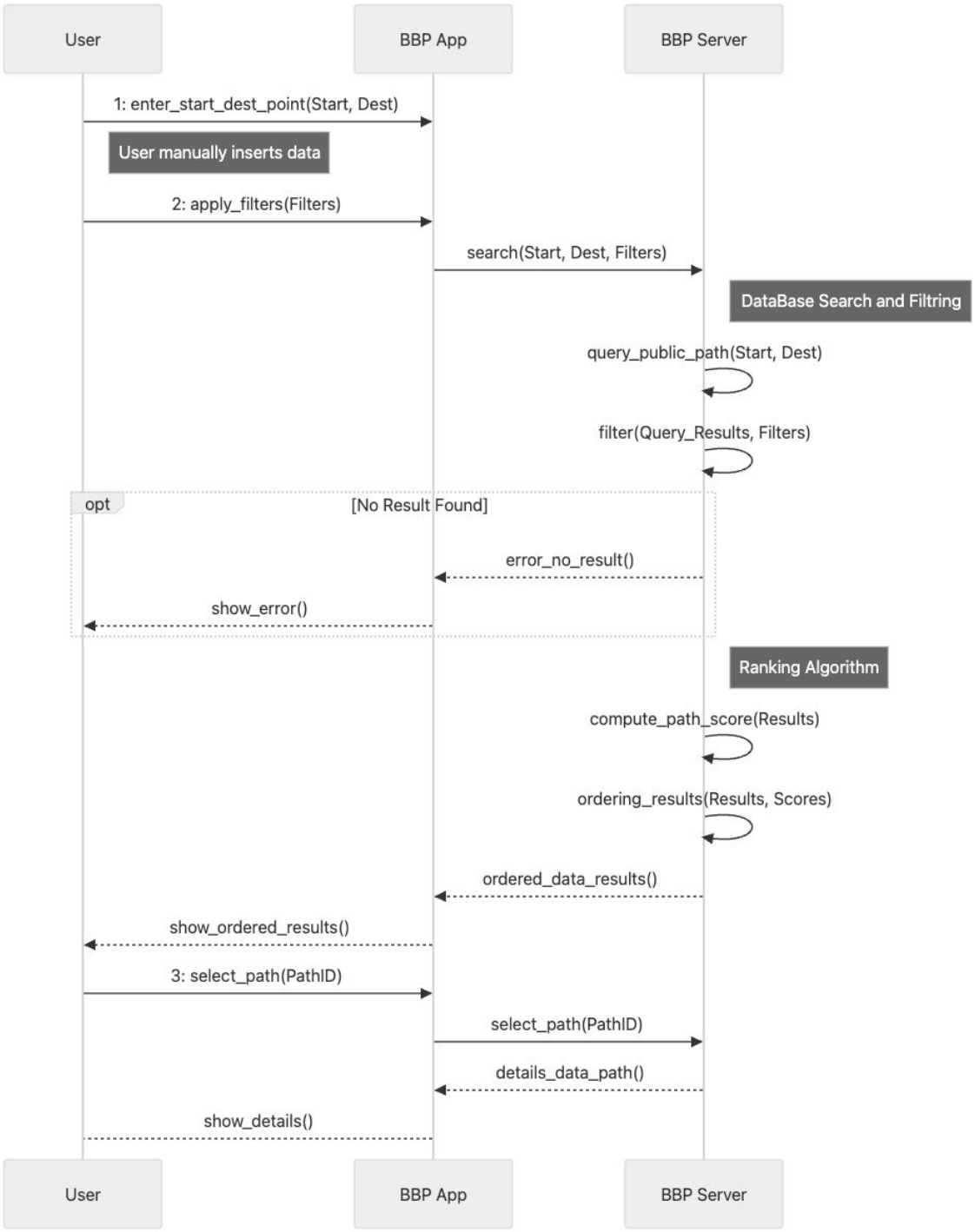


Figure 3.13: [UC7] Search for a path Sequence Diagram

3.2.4 Requirement Mapping

The following tables map the project goals to the specific requirements, domain assumptions (A), and dependencies (D) necessary to achieve them.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

G1: Support cyclists in tracking their activities	
Requirements	**Assumptions & Dependencies**
R1: The platform allows users to create an account by providing the required personal information. **R2**: The platform allows registered users to log in and access the system. **R3**: The platform allows registered users to recover the access credentials. **R4**: The platform allows guest users (non-registered users) to access only the path search functionality of the system. **R5**: The system shall allow registered users to start recording a new cycling activity. **R6**: The system shall allow users to select an existing path from the path public repository to follow during the activity (not mandatory). **R7**: The system shall collect raw data from the mobile device's sensors (GPS, accelerometer, gyroscope) during the recording of an activity. **R8**: The system shall analyze the user's speed derived from GPS data to verify if the user is effectively cycling. **R9**: The system shall analyze accelerometer and gyroscope data to detect potential obstacles or road surface anomalies. **R10**: The system shall calculate real-time statistics during the recording, including total distance covered and average speed. **R11**: The system shall allow the user to pause and resume the recording of an activity.	**A3**: **Recording Implies Path Data:** It is assumed that recording a sporting activity implicitly involves collecting path data (GPS/Status) for potential path creation. **D1**: **External weather service:** The system relies on the availability and accuracy of GPS data from users' devices to record cycling activities and create paths. **D3**: **Storage database:** Stores all platform data, including user routes, additional information (road conditions, obstacles), user profiles and publication metadata. Ensures persistence, consistency, and efficient retrieval.

Table 3.16: Requirement Mapping for G1

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

G2: Allow cyclists to contribute information about bike paths	
Requirements	**Assumptions & Dependencies**
R12: Upon stopping an activity, the system shall display the list of automatically detected obstacles. **R13**: The system shall require the user to validate the detected obstacles before proceeding. **R14**: After validation, the system shall analyze the recorded GPS trace to identify if it matches an existing path in the database. **R15**: If the recorded trace matches an existing public path, the system shall automatically associate the activity with that path entity. **R16**: The system shall use the validated data to update the status and information of the existing public path. **R17**: The system shall add the identified path to the "My paths" section if not already present. **R18**: If the recorded trace doesn't match any existing public path, the system shall allow users to generate a new path entity starting from the GPS trace of a recorded activity. **R19**: The system shall allow the user to associate a name to the recorded path and store it in the "My paths" section. **R20**: The system shall allow the user to create a new path manually by selecting a sequence of streets from a map. **R21**: The system shall require the user to manually input the status of the streets composing the path and mark the position and type of obstacles along it. **R22**: The system shall allow the user to assign a name to the manually created path and store it in the "My Paths" section. **R26**: The platform is able to detect when a registered user follows a path that is already available to the public and associate the activity with that path. **R27**: The platform allows registered users to update the condition information of a path, even without performing an activity on that path.	**A2**: **Validation Constraint:** While user confirmation can happen anytime for private activities, automatically detected data (sensors) must be validated by the user before being used for public updates. **A5**: **Manual Creator Competence:** It is assumed that when a user creates a path manually, they have physically traversed it and possess knowledge of its current status. **D5**: **Icon service:** Provides a library of graphical icons used across the interface for consistent and intuitive representation of road conditions, obstacles, route statuses, and other UI elements.

Table 3.18: Requirement Mapping for G2

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

G3: Enable the publication of recorded path information	
Requirements	**Assumptions & Dependencies**
R28: The system shall allow users to edit the information (name, street status, obstacles) of any path they have created. **R29**: The system shall allow users to publish a stored path, making it available to the community. **R12**: Upon stopping an activity, the system shall display the list of automatically detected obstacles. **R13**: The system shall require the user to validate the detected obstacles before proceeding. **R14**: After validation, the system shall analyze the recorded GPS trace to identify if it matches an existing path in the database. **R15**: If the recorded trace matches an existing public path, the system shall automatically associate the activity with that path entity. **R16**: The system shall use the validated data to update the status and information of the existing public path. **R17**: The system shall add the identified path to the "My paths" section if not already present. **R18**: If the recorded trace doesn't match any existing public path, the system shall allow users to generate a new path entity starting from the GPS trace of a recorded activity. **R19**: The system shall allow the user to associate a name to the recorded path and store it in the "My paths" section. **R20**: The system shall allow the user to create a new path manually by selecting a sequence of streets from a map. **R21**: The system shall require the user to manually input the status of the streets composing the path and mark the position and type of obstacles along it. **R22**: The system shall allow the user to assign a name to the manually created path and store it in the "My Paths" section.	**A4**: **Contribution via Merging:** Once a path is public, its core geometry cannot be overwritten; updates happen through feedback/reports that are merged by the system. **A6**: **Path Uniqueness:** The system assumes that no two identical public paths (same geometry) can exist in the database. **A7**: **Manual Duplicate Avoidance:** Automatic duplicate detection relies on GPS tracks. For manual creation, it is assumed the user checks the map for duplicates before creating a new path. **D4**: **Authentication services:** Manages user identity verification, login processes and secure access to personal data and features.

Table 3.20: Requirement Mapping for G3

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

G4: Enable route search and visualization	
Requirements	**Assumptions & Dependencies**
R23: The platform allows users to search for cycling paths based on a starting point and an ending point. **R24**: The platform allows users to filter the search results based on the path parameters. **R25**: The platform is able to rank the available paths based on the path data, including the condition of each segment, the freshness of the information, and the effectiveness of the path in connecting the selected points. **R1**: The platform allows users to create an account by providing the required personal information. **R2**: The platform allows registered users to log in and access the system. **R3**: The platform allows registered users to recover the access credentials. **R4**: The platform allows guest users (non-registered users) to access only the path search functionality of the system.	**A1**: **Path Optimality Definition:** A path is considered "better" than another if it minimizes the travel distance or if its start/end points are closer to the user's desired origin/destination. **D2**: **Mapping service:** Offers geographic data and map layers used to visualize routes, match GPS tracks to road segments and retrieve details about paths, intersections, and terrains.

Table 3.22: Requirement Mapping for G4

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

G5: Maintain up-to-date and reliable path information	
Requirements	**Assumptions & Dependencies**
R12: Upon stopping an activity, the system shall display the list of automatically detected obstacles. **R13**: The system shall require the user to validate the detected obstacles before proceeding. **R14**: After validation, the system shall analyze the recorded GPS trace to identify if it matches an existing path in the database. **R15**: If the recorded trace matches an existing public path, the system shall automatically associate the activity with that path entity. **R16**: The system shall use the validated data to update the status and information of the existing public path. **R17**: The system shall add the identified path to the "My paths" section if not already present. **R18**: If the recorded trace doesn't match any existing public path, the system shall allow users to generate a new path entity starting from the GPS trace of a recorded activity. **R19**: The system shall allow the user to associate a name to the recorded path and store it in the "My paths" section. **R26**: The platform is able to detect when a registered user follows a path that is already available to the public and associate the activity with that path. **R27**: The platform allows registered users to update the condition information of a path, even without performing an activity on that path. **R25**: The platform is able to rank the available paths based on the path data, including the condition of each segment, the freshness of the information, and the effectiveness of the path in connecting the selected points.	**A4**: **Contribution via Merging:** Once a path is public, its core geometry cannot be overwritten; updates happen through feedback/reports that are merged by the system. **A2**: **Validation Constraint:** While user confirmation can happen anytime for private activities, automatically detected data (sensors) must be validated by the user before being used for public updates. **D3**: **Storage database:** Stores all platform data, including user routes, additional information (road conditions, obstacles), user profiles and publication metadata. Ensures persistence, consistency, and efficient retrieval.

Table 3.24: Requirement Mapping for G5

3.3 Performance Requirements

The Best Bike Paths (BBP) system must ensure high performance to guarantee a smooth user experience and reliable tracking reliability. Regarding response time, the backend must compute path scores and visualize routing results within 3 seconds for 95% of requests, supporting a concurrent load of at least 1,000 active users. On the mobile client side, energy efficiency is a critical requirement: during 'Automated mode', the combined use of GPS and sensors must not exceed a battery consumption rate of 15% per hour on standard devices. Finally, to ensure the relevance of shared information (data freshness), path status updates derived from community reports must be propagated and visible to users within 15 minutes of processing.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3.4 Design Constraints

3.4.1 Standards Compliance

- **GDPR Compliance (EU 2016/679):** Since the system collects and processes sensitive personal data (specifically geolocation history and user performance metrics), the architecture must be fully compliant with the General Data Protection Regulation (GDPR). This requires the implementation of explicit consent mechanisms, data anonymization where possible, and the "Right to be Forgotten".
- **ISO/IEC 27001 (Information Security):** The system design must adhere to ISO/IEC 27001 principles for Information Security Management. This constrains the architecture to include mandatory security controls, such as end-to-end encryption (TLS/WSS) for data in transit and encryption for sensitive data at rest.
- **Geospatial Standards (WGS84 & GPX):** To guarantee global interoperability and compatibility with external mapping services, all geospatial coordinates must be stored using the WGS84 reference system. Additionally, data portability for bike paths must support the GPX (GPS Exchange Format) standard.
- **Mobile UI/UX Guidelines:** The mobile application client must strictly follow the Human Interface Guidelines (for iOS) and Material Design principles (for Android). This ensures a user experience that is consistent with the native operating system environment.

3.4.2 Hardware Limitations

- **Sensor Suite Requirements:** The availability of the "Automated Mode" is strictly bound by the hardware capabilities of the user's device. The application requires the mandatory presence of:
  - A functional GNSS/GPS Receiver.
  - An Accelerometer.
  - A Gyroscope.
- **Operating System Compatibility:** To ensure the reliability of sensor data collection (GPS, Accelerometer, Gyroscope) and compliance with modern security standards for background location tracking, the client application must support:
  - Android: Version 10.0 (API Level 29) or higher (covering approx. 89% of the active market).
  - iOS: Version 15.0 or higher (covering approx. 90% of the active market).

3.4.3 Any other Constraint

- **Safety & Distraction-Free UI:** To comply with traffic safety regulations and ensure user well-being, the "Automated Mode" interface must be designed as interaction-free. The application shall not prompt for user input or require manual interaction while the device detects that the user is cycling.

3.5 Software System Attributes

3.5.1 Reliability

The system must ensure the integrity and preservation of data collected from mobile sensors in a dynamic and potentially unstable environment.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

- **Data Persistence:** The mobile application must implement local data caching to handle network instability. It should store activity data temporarily on the device and synchronize it with the central server once a stable connection is re-established, preventing data loss during rides in remote areas.
- **Operational Safety:** Reliability also extends to the safe usage of the application. The system must provide a non-distracting user experience to minimize cognitive load while cycling, ensuring the software aids rather than hinders the user's physical safety.
- **Backend Stability:** On the server side, the database should employ replication strategies to prevent data loss. The merging algorithm must reliably handle concurrent updates from multiple users, maintaining a consistent state of the path quality scores.

3.5.2 Availability

The BBP platform should deliver a highly available and responsive service.

- **Service Continuity:** Core functionalities, such as activity recording and path visualization, must incorporate redundancy and failover mechanisms. The architecture must handle failures of the external meteorological service, allowing the system to continue saving trips rather than rejecting the user's activity.
- **Responsiveness:** Availability implies immediate reaction times. Map rendering and navigation cues must be generated with minimal latency to ensure the user receives timely directions while moving at speed.
- **Uptime:** The design should aim to maximize uptime, performing maintenance updates during low-traffic periods to avoid disrupting users during peak cycling times.

3.5.3 Security

The system handles sensitive user data, such as location history and movement patterns, which requires strict protection measures.

- **Privacy & Anonymization:** Given the sensitivity of GPS tracking, the system should adopt measures to anonymize data when aggregating it for public path statistics, ensuring that individual routes cannot be reverse-engineered to identify specific users.
- **Data Protection:** Authentication must be robust. All stored data must be encrypted at rest and communication between the mobile app and the server must be secured via HTTPS/SSL to prevent interception.
- **Access Control:** The system must implement strict access control policies, ensuring that private paths in the "My Paths" section are accessible only to their owners until explicitly published.

3.5.4 Maintainability

The components should be designed to be modular and easily scalable.

- **Modularity:** The system should decouple the core logic from the interface layers. This allows developers to update or optimize specific algorithms without affecting the rest of the application.
- **Documentation & Logging:** The code must be fully documented, including API definitions. The system should implement comprehensive logging, particularly for the data synchronization and merging processes, to facilitate rapid debugging and maintenance.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3.5.5 Portability

The system must be compatible with a wide range of devices and environments, respecting their hardware constraints.

- **Resource Efficiency:** True portability on mobile devices requires optimizing resource usage. The application must implement adaptive sensor sampling to minimize battery drain (e.g., reducing GPS polling when stationary) and data batching to reduce network bandwidth usage, ensuring the app is usable even on older devices or with limited data plans.
- **Cross-Platform Compatibility:** The mobile application must function consistently across major operating systems, abstracting hardware differences in sensors (accelerometers/gyroscopes).
- **Web Accessibility:** The web interface must be compatible with all major browsers (Chrome, Firefox, Safari) and responsive to various screen sizes.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 4

Formal Analysis Using Alloy

This chapter presents the formal specification and verification of the *BestBikePaths* ecosystem using the Alloy Analyzer. The primary objective of this analysis is to validate the structural consistency and the domain logic governing the transition from raw sensor data to usable paths.

We defined a set of *Signatures* to represent the core entities and *Facts* to enforce strict system invariants. Finally, the robustness of the model was tested through three specific predicates (scenarios):

1. **Full Lifecycle:** Verifying the complete flow from data recording to public feedback.
2. **Conflicting Feedback:** Ensuring the system correctly handles divergent opinions on the same path.
3. **Safety Filtering:** Demonstrating the automatic rejection of invalid activities (e.g., walking speed) to prevent system pollution.

The following sections detail the model implementation and the results of the executed instances.

```
1  module BestBikePathsUnified
2
3  /*
       ================================================================================
4      1. DATA TYPES AND ENUMERATIONS
5      Defines basic domains: geographic locations, visibility states,
6      road surface conditions, and boolean values for flags.
7
       ================================================================================
        */
8  sig Location {}
9
10 enum PathStatus { Private, Public }
11 enum Condition { Optimal, Sufficient, Bad }
12 enum Bool { True, False }
13
14 /*
       ================================================================================
15     2. SYSTEM ACTORS
16     The User is the central entity who owns activities (recordings),
17     paths (publications), and leaves feedback.
18
       ================================================================================
        */
19 sig User {
20     myActivities: set Activity,
21     myPaths: set Path,
```

```
    myFeedbacks: set Feedback
}

/*
    ================================================================================

   3. CORE ENTITIES

    ================================================================================
     */

/* ENTITY: ACTIVITY (Sensing/Partner Logic)
   Represents raw sensor data recording.
   - trace: the GPS sequence.
   - avgSpeed: used to filter if the activity is actually "biking".
   - sensorAnomalies: potholes automatically detected by accelerometer.
*/
sig Activity {
    owner: one User,
    trace: seq Location,
    avgSpeed: Int,
    sensorAnomalies: set Location
}

/* ENTITY: PATH (Structure and Publishing Logic)
   Represents the consolidated path in the system.
   - trajectory: the geometry of the path.
   - status/isConfirmed: manage the lifecycle (draft -> public).
   - currentCondition: the aggregated state of the path quality.
*/
sig Path {
    owner: one User,
    trajectory: seq Location,
    status: one PathStatus,
    isConfirmed: one Bool,
    currentCondition: one Condition,
    receivedFeedbacks: set Feedback
}

/* ENTITY: FEEDBACK (Community Logic)
   Represents user opinion on a public path.
   Links an author to a target path, specifying conditions and localized
    obstacles.
*/
sig Feedback {
    author: one User,
    targetPath: one Path,
    reportedCondition: one Condition,
    reportedObstacles: set Location
}

/*
    ================================================================================

   4. FACTS (INVARIANTS AND DOMAIN RULES)

    ================================================================================
     */

/* FACT: STRUCTURAL CONSISTENCY
   Ensures bidirectional relationships are consistent (e.g., if feedback
   belongs to a user, it must appear in that user's feedback list)
```

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

```
   and establishes ownership hierarchy.
*/
fact StructuralConsistency {
    all a: Activity | a in a.owner.myActivities
    all p: Path | p in p.owner.myPaths
    all f: Feedback | f in f.author.myFeedbacks
    all f: Feedback | f in f.targetPath.receivedFeedbacks
}

/* FACT: GEO CONSISTENCY
   Constrains spatial data:
   1. Obstacles reported in feedback must be on the path trajectory.
   2. Anomalies detected by sensors must be on the activity trace.
   Note: .elems converts the sequence to a set for inclusion check.
*/
fact GeoConsistency {
    all f: Feedback | f.reportedObstacles in f.targetPath.trajectory.elems
    all a: Activity | a.sensorAnomalies in a.trace.elems
}

/* FACT: UNIQUE FEEDBACK (Anti-Spam)
   A user can leave at most one feedback per specific path.
   This prevents ranking manipulation by a single actor.
*/
fact UniqueFeedback {
    all p: Path, u: User | lone f: p.receivedFeedbacks | f.author = u
}

/* FACT: ACTIVITY VALIDATION LOGIC (Sensing)
   Defines rules for an activity to become a path:
   If a path is generated from an activity (sharing owner and trace),
   then the average speed must be compatible with biking
   (here defined between 4 and 12 to stay within Alloy's small integer range).
*/
fact ActivityToPathLogic {
    all p: Path, a: Activity |
        (p.trajectory = a.trace and p.owner = a.owner) implies {
            a.avgSpeed > 4
            a.avgSpeed < 12
    }
}

/* FACT: PRIVATE PATH PRIVACY
   If a path is set to Private, no one except the owner can leave feedback
   or interact with it.
*/
fact PrivatePathPrivacy {
    all f: Feedback |
        f.targetPath.status = Private implies f.author = f.targetPath.owner
}
```

Listing 4.1: Complete Alloy Model for BestBikePaths

4.1 Full Lifecycle Scenario

```

/*
    ================================================================================
   5. PREDICATES (TEST SCENARIOS)
```

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

```
4
    ================================================================================
     */
5
6  /* SCENARIO 1: THE FULL LIFECYCLE
7     Demonstrates the ideal "Happy Path":
8     1. A user records a valid activity (correct speed, anomalies detected).
9     2. The system generates a path from this activity.
10    3. The user confirms and publishes the path.
11    4. A different user leaves feedback on the path.
12 */
13 pred FullLifecycleScenario {
14     some u1, u2: User | some a: Activity | some p: Path | some f: Feedback | {
15         a.owner = u1
16         a.avgSpeed = 6
17         #a.sensorAnomalies > 0
18
19         p.owner = u1
20         p.trajectory = a.trace
21
22         p.isConfirmed = True
23         p.status = Public
24
25         f.author = u2
26         u1 != u2
27         f.targetPath = p
28         f.reportedCondition = Bad
29
30         #p.trajectory > 2
31     }
32 }
33 run FullLifecycleScenario for 5 but 5 Int
```

Listing 4.2: Full Lifecycle Scenario Alloy Model

This scenario represents the **"Happy Path"** of the *BestBikePaths* ecosystem. The objective is to validate the complete data flow across the three main layers of the system: **Sensing** (data collection), **Publishing** (path creation), and **Community** (user review).

The predicate constrains the Alloy Analyzer to generate an instance where:

1. **User A** records an activity with a valid biking speed (simulated here as 6 units, satisfying the > 4 threshold).
2. The system successfully promotes this activity to a **Public Path**, preserving the GPS trace.
3. **User B** (distinct from User A) retrieves the path and submits a **Feedback**.

4.1.1 Visual Analysis

The following figures illustrate the specific instance generated by Alloy that satisfies this predicate.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

1. The Global Instance (World View)

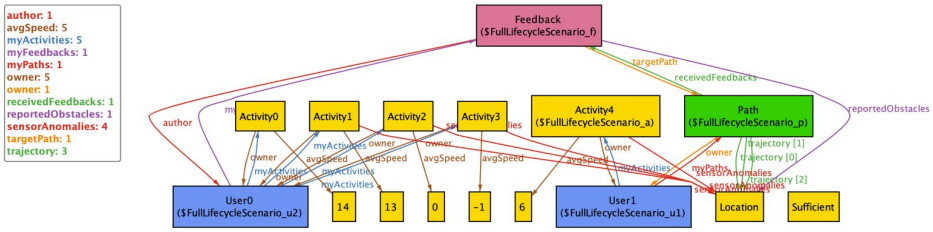


Figure 4.1: Full Lifecycle Scenario Graph

Figure 4.1 provides a holistic view of the generated universe, identifying the actors and their relationships:

- **The Actors:** Two distinct users are present. `User1` (bottom right) acts as the **Creator**, while `User0` (bottom left) acts as the **Reviewer**.
- **The Connection:** `User1` owns `Activity4` (yellow), which is the source of the generated `Path` (green).
- **The Interaction:** The `Feedback` node (pink) targets the `Path` and links back to `User0` as the author. This visually confirms that the review comes from the community, not the owner, satisfying the structural integrity constraints.

2. Constraint Verification (Detail View)

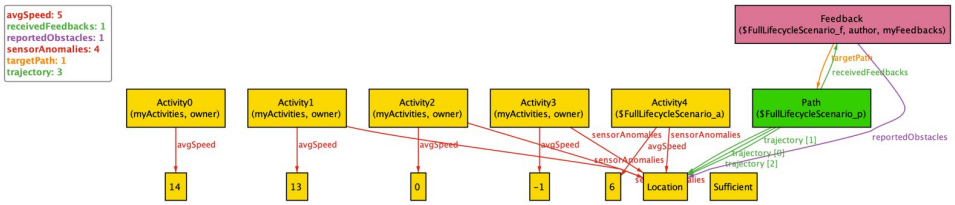


Figure 4.2: Full Lifecycle Scenario Graph, Projection User0.

Figure 4.2 filters out the actors to focus on data validity and numeric constraints:

- **Speed Threshold:** The central entity, `Activity4` (highlighted in large yellow), shows an `avgSpeed` of **6**. This explicitly validates the `ActivityToPathLogic` fact (which requires speed > 4), allowing the creation of the green `Path`.
- **Unused Atoms:** Note that other activities (`Activity0`, 1, 2, 3) exist in the universe with various speeds (e.g., 14, 13, 0, -1). However, they remain disconnected from any Path because they were not selected by the predicate logic, demonstrating the model's ability to distinguish between valid and irrelevant data.
- **Data Preservation:** The arrows for `sensorAnomalies` connect both the Activity and the Path to the same `Location` node, proving that safety data (e.g., potholes) is correctly inherited during publication.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

3. The Lifecycle Projection (Data Flow)

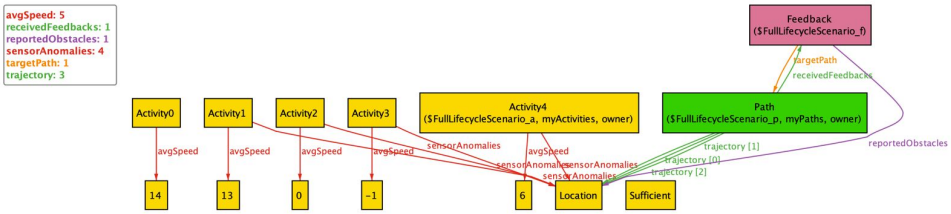


Figure 4.3: Alloy instance: Full Lifecycle Scenario Projection User1.

Figure 4.3 offers the clearest representation by projecting the graph over the data evolution timeline (hiding the user nodes). It depicts the linear progression of information:

1. **Input (Sensing):** The flow begins on the left with `Activity4`.
2. **Transformation (System):** The activity is transformed into the confirmed `Path` (center, green).
3. **Output (Community):** The path receives a `Feedback` (right, pink).

This sequence—`Activity` → `Path` → `Feedback`—visually proves that the model successfully supports the entire lifecycle of a crowd-sourced bike path without logical interruptions.

4.2 Conflicting Feedback Merge Scenario

```
/* SCENARIO 2: CONFLICTING FEEDBACK MERGE
Verifies that the system supports divergent opinions.
Generates a public path with at least two feedbacks reporting
opposite conditions (e.g., Optimal vs Bad).
*/
pred ConflictingFeedbackMerge {
    some p: Path | {
        p.status = Public
        #p.receivedFeedbacks >= 2
        some disj f1, f2: p.receivedFeedbacks | {
            f1.reportedCondition = Optimal
            f2.reportedCondition = Bad
        }
    }
}
run ConflictingFeedbackMerge for 4 but 5 Int
```

Listing 4.3: Conflicting Feedback Merge Scenario

This scenario demonstrates the system's capability to handle and persist divergent opinions from the community regarding the same public path, a necessary condition for the merging logic. The generated instance depicts a single Path with Public status linked to multiple distinct Feedback entities. Specifically, the model enforces the existence of conflicting reports, where one user rates the condition as Optimal while another rates it as Bad. This scenario formally validates the structural prerequisites for the Data Merging requirement. It proves that the domain model allows for the coexistence of contradictory data points without inconsistency. This accumulation of diverse feedback is the input required by the server-side aggregation algorithm to dynamically compute the path's overall score and `currentCondition`.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

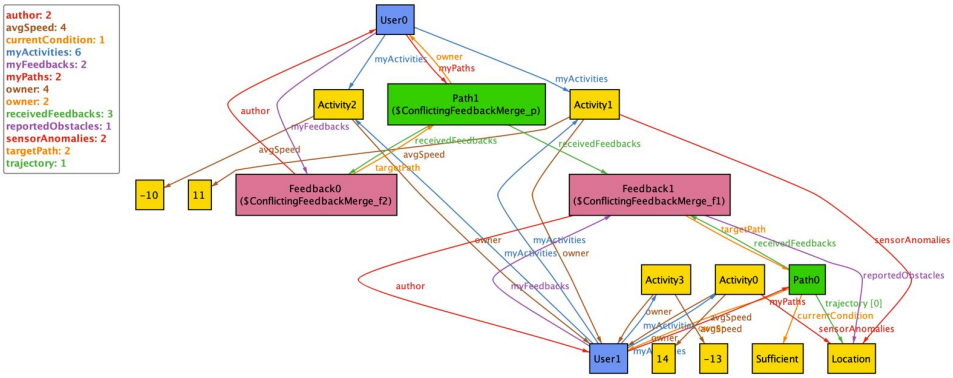


Figure 4.4: Conflicting Feedback Merge Scenario Graph

4.3 Rejected Activity Filter Scenario

```
/* SCENARIO 3: REJECTED ACTIVITY FILTER
   Demonstrates system robustness (Safety).
   Verifies the existence of an activity that is too slow (e.g., walking at
    speed 3)
   and ensures that NO path is generated from it.
*/
pred RejectedActivityScenario {
    some a: Activity | {
        a.avgSpeed = 3

        no p: Path | p.owner = a.owner and p.trajectory = a.trace
    }
}
run RejectedActivityScenario for 3 but 5 Int
```

Listing 4.4: Rejected Activity Filter Scenario

This scenario serves as a negative test to verify the robustness of the system's filtering logic, ensuring that invalid activities do not pollute the path database. The predicate simulates an edge case where an Activity is recorded with an `avgSpeed` of 3 (representing a walking pace), which falls below the mandatory threshold defined in the domain facts (speed >4). The solver confirms that, in this state, no Path entity sharing the same trace and owner exists. This successfully verifies the Safety and Data Integrity constraints. It demonstrates that the logical rules correctly prevent the promotion of non-cycling activities (e.g., walking or running) into public paths, thereby guaranteeing that the platform remains focused exclusively on biking data.

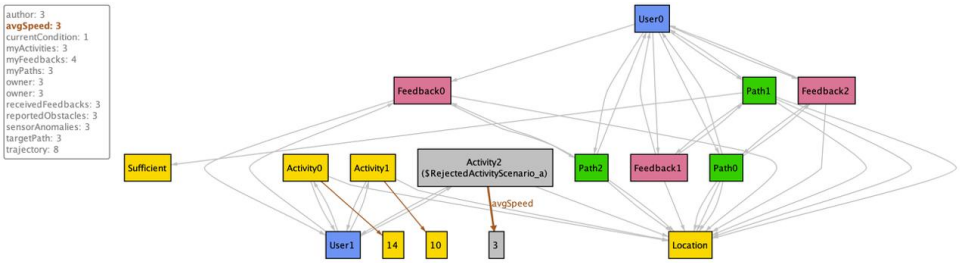


Figure 4.5: Rejected Activity Scenario Graph

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved

Chapter 5

Effort Spent

Chapter	Date	Alessio Pesavento	Alberto Spezzapria	Luca Suzani
Introduction	27/10/2025	1h	1h	1h
	07/11/2025	2h	2h	2h
	08/11/2025	-	1h	-
	09/11/2025	1h	-	-
	10/11/2025	-	-	1h
	Total	**4h**	**4h**	**4h**
Overall Description	11/11/2025	1.5h	1.5h	1.5h
	12/11/2025	-	1h	-
	13/11/2025	1.5h	1h	2h
	15/11/2025	1h	1h	-
	22/11/2025	2h	2h	2h
	25/11/2025	1h	1h	1h
	28/11/2025	2h	2h	2h
	29/11/2025	-	1h	1h
	01/12/2025	2h	2h	3h
	11/12/2025	2.5h	-	-
	Total	**13.5h**	**12.5h**	**12.5h**
Specific Requirements	03/12/2025	3h	3h	3h
	05/12/2025	3h	3h	3h
	08/12/2025	4h	4h	4h
	09/12/2025	-	0.5h	1.5h
	10/12/2025	2h	2h	-
	11/12/2025	-	1.5h	1h
	12/12/2025	-	1.5h	1h
	14/12/2025	2h	2h	2h
	16/12/2025	3h	3h	3h
	19/12/2025	0.5h	-	-
	Total	**17.5h**	**20.5h**	**18.5h**
Formal Analysis Using Alloy	13/12/2025	2h	2h	2h
	15/12/2025	3h	3h	3h
	17/12/2025	1h	-	-
	18/12/2025	2h	2h	2h
	19/12/2025	-	2h	-
	Total	**8h**	**9h**	**7h**
TOTAL		**43h**	**46h**	**42h**

Table 5.1: Hours of work spent by each author redacting the RASD document

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved